

Duale Hochschule Baden-Württemberg Mannheim

Studienarbeit

Übersetzung von Vorlesungsunterlagen in TypeScript – Theoretische Informatik III

Studiengang Informatik

Studienrichtung Angewandte Informatik

Verfasser:	Ertan Ertas und Christian Strerath
Matrikelnummer:	9390224, 4965013
Kurs:	TINF23AI2
Studiengangsleiter:	Prof. Dr. Holger Hofmann
Betreuer:	Prof. Dr. Karl Stroetmann
Bearbeitungszeitraum:	15.10.2025 - 14.04.2026

Erklärung

Wir versichern hiermit, dass wir unsere Studienarbeit mit dem Thema: „Übersetzung von Vorlesungsunterlagen in TypeScript – Theoretische Informatik III“ selbstständig verfasst und keine anderen als die angegebenen Quellen und Hilfsmittel benutzt haben.

Mannheim, 8. April 2026

Ertan Ertas

Kaiserslautern, 8. April 2026

Christian Strerath

Kurzfassung (Abstract)

Auf Deutsch

In English

Inhaltsverzeichnis

1	Einleitung	1
1.1	Problemstellung	1
1.2	Zielsetzung	2
1.3	Methodik und Abgrenzung	3
2	Theoretische Grundlagen der Typisierung	5
2.1	Semantik und Funktion von Datentypen	5
2.2	Statische Typisierung mittels TypeScript	7
2.2.1	Herausforderungen einer naiven Code-Migration	8
2.2.2	Explizite Typannotationen	11
2.2.3	Strukturelles Typing und Interface-Verträge	17
2.3	Fazit zur architektonischen Neuausrichtung	23
3	RecursiveSet: API und fachliche Modellierung	26
3.1	Mathematische Mengen und Wohlunterscheidbarkeit	26
3.2	Kernbausteine der Bibliothek	27
3.2.1	Die Datenstruktur RecursiveSet	27
3.2.2	Strukturierte Elemente: Die Tuple-Klasse	28
3.2.3	Assoziative Daten: Die RecursiveMap	29
3.3	Verarbeitung und Lebenszyklus strukturierter Daten	30
3.3.1	Der Freeze-Contract: Immutabilität sichern	30
3.3.2	Klassische Mengenoperationen	31
3.3.3	Teil- und Obermengenbeziehungen	32
3.3.4	Auswahl eines beliebigen Elements	33
3.3.5	Kartesisches Produkt und Potenzmenge	33
3.3.6	Funktionale Transformationen statt Komprehension	35
4	Interne Architektur von RecursiveSet	38
4.1	Architektonische Grundentscheidungen	38
4.2	Die Hash-Engine (Zentrale Komponente)	39
4.2.1	Ganzzahlen und der ArrayBuffer-Trick für IEEE-754	39
4.2.2	Hashing von Strings und strukturellen Objekten	42
4.3	Speicherlayout (Structure of Arrays)	44
4.4	Konzepte der Gleichheit und Type Narrowing	46
4.5	Kollisionsauflösung und Index-Verwaltung	48
4.5.1	Open Addressing mit Linear Probing	48
4.5.2	Löschvorgang: Tombstones und Backshift Deletion	50
4.6	Dynamische Kapazitätsverwaltung (Resizing und Rehashing)	53

4.7	Technische Umsetzung des Freeze-Contracts	55
5	Vom Python-Werkzeug PLY zu Lezer	57
5.1	Grundlagen von Lezer	58
5.2	Grammatikdefinition in Lezer	60
5.3	Generierung des Parsers	61
5.4	Integration in die TypeScript-Implementierung	63
6	Case Study: Umsetzung eines vollständigen Interpreters in TypeScript	66
6.1	Ziel und Aufbau des Notebooks	66
6.2	Sprachdefinition im Vergleich: Python (PLY) vs. TypeScript (Lezer)	67
6.3	Transformation vom CST zum AST	71
6.4	Darstellung rekursiver Listenstrukturen	71
6.5	Typisierung und interne AST-Repräsentation	76
6.6	Interpreterarchitektur im Vergleich	77
6.7	Auswertung arithmetischer Ausdrücke	77
6.8	Auswertung boolescher Bedingungen	81
6.9	Umgebungsmodell und Programmausführung	81
6.10	Ergebnisse der Case Study	83
7	Fazit	84
8	Literaturverzeichnis	86
A	Anhang	87
A.1	Verzeichnis der portierten Lehrnotebooks	87

Abbildungsverzeichnis

Tabellenverzeichnis

1	Übersicht der übersetzten Notebooks im Repository	87
---	---	----

List of Listings

1	Mengen in Python: Das erwartete Verhalten	2
2	Mengen in TypeScript: Das fehlerhafte Verhalten	2
3	Unterschiedliche Operationen erfordern unterschiedliche Typen . .	5
4	Dynamische Typisierung in Python	6
5	Dynamische Typisierung in JavaScript	6
6	Typfehler wird erst zur Laufzeit sichtbar	6
7	Eine untypisierte Funktion in JavaScript	7
8	Dieselbe Funktion in TypeScript mit expliziten Typen	7
9	Naive Übersetzung einer Funktion nach TypeScript	8
10	Typprüfung zur Laufzeit in Python	9
11	Typprüfung zur Laufzeit in JavaScript und TypeScript	9
12	Der Typ <code>any</code> hebt die Typprüfung aus	9
13	Explizite Umwandlung in Python	10
14	Implizite und explizite Umwandlung in JavaScript	10
15	Eine Type Assertion ist keine echte Umwandlung	10
16	Unpräzise vs. präzise formulierte Funktion in TypeScript	11
17	Atomare und zusammengesetzte TypeScript-Typen	12
18	Union Type aus Literal Types in TypeScript	13
19	Literal Types in Python	13
20	Ein möglicher fehlender Rückgabewert in TypeScript	14
21	Möglicher fehlender Rückgabewert in Python	14
22	Fehlerhafte Nutzung ohne vorherige Prüfung	15
23	Explizite Behandlung eines möglichen <code>undefined</code> -Werts	15
24	Der Typ <code>unknown</code> erzwingt eine vorherige Prüfung	16
25	Einengung von <code>unknown</code> durch Typprüfung im Kontrollfluss	16
26	Type Hints in Python	16
27	Python <code>Any</code> vs. TypeScript <code>unknown</code>	17
28	Ein Interface als Vertrag über benötigte Methoden	18
29	Eine Klasse erfüllt ein Interface über ihre reine Struktur	18
30	Ein Protocol in Python als statischer Vertrag	19
31	Feste Datenstruktur mit <code>TypedDict</code> in Python	19
32	Objektstrukturen und Aliase: <code>interface</code> und <code>type</code> in TypeScript . .	20
33	Eine generische Funktion in TypeScript	21
34	Eine generische Funktion in Python	21
35	Generics mit strukturellen Schranken im Vergleich	22
36	Type Narrowing durch Kontrollfluss und <code>typeof</code>	22
37	Klassenprüfung mit <code>instanceof</code>	23

38	Vollständige Fallunterscheidung mit <code>never</code>	24
39	Mengenverhalten in Python	26
40	Mengenverhalten in TypeScript	27
41	Hinzufügen von Elementen in Python und mit <code>RecursiveSet</code>	28
42	Mitgliedschaftstest in Python und mit <code>RecursiveSet</code>	28
43	Entfernen von Elementen in Python und mit <code>RecursiveSet</code>	29
44	Roboter-Koordinaten: Tupel in Python vs. TypeScript	29
45	Dictionaries in Python vs. <code>RecursiveMap</code> in TypeScript	30
46	Der Freeze-Contract bei geschachtelten Mengen	31
47	Arbeiten mit einer veränderlichen Kopie	31
48	Vereinigung zweier Mengen in Python und mit <code>RecursiveSet</code>	32
49	Schnittmenge zweier Mengen in Python und mit <code>RecursiveSet</code>	32
50	Differenz zweier Mengen in Python und mit <code>RecursiveSet</code>	32
51	Symmetrische Differenz in Python und mit <code>RecursiveSet</code>	33
52	Teil- und Obermengenbeziehungen im Sprachvergleich	33
53	Auswahl eines Elements in Python und mit <code>RecursiveSet</code>	34
54	Kartesisches Produkt in Python und mit <code>RecursiveSet</code>	34
55	Potenzmenge mit <code>RecursiveSet</code>	34
56	Python: Mengenkompensation mit Selektion und Transformation	35
57	Selektion mit <code>filter</code>	35
58	Abbildung mit <code>map</code>	36
59	Logische Quantoren mit <code>every</code> und <code>some</code>	36
60	Kombination aus Filter und Abbildung mit <code>filterMap</code>	36
61	Python: Verschachtelte Mengenkompensation	37
62	Verschachtelte Konstruktion mit <code>flatMap</code>	37
63	Aggregation mit <code>reduce</code>	37
64	Hashing von numerischen Werten	40
65	Hashing von Strings und strukturellen Objekten	42
66	Der Vertrag für komplexe Mengenelemente	43
67	Speicherarchitektur im Structure of Arrays (SoA) Format	44
68	Die <code>equals</code> -Methode von <code>RecursiveSet</code>	47
69	Linear Probing am Beispiel der <code>has</code> -Methode	49
70	Reparatur der Sondierungskette (Backshifting)	52
71	Der Rehashing-Prozess bei einer Kapazitätsänderung	55
72	Einfrieren durch Hash-Abruf	56
73	Ply-Grammatik in Python	58
74	Lezer-Grammatik in TypeScript	58
75	Umsetzung Lezer-Grammatik in TypeScript	61
76	Erstellung des Parsers in TypeScript	62

77	ParseStmnt Funktion in TypeScript	64
78	Ply-Grammatik	68
79	Lezer-Grammatik-Teil1	69
80	Lezer-Grammatik-Teil2	70
81	Direkte AST-Konstruktion in PLY Teil1	72
82	Direkte AST-Konstruktion in PLY Teil2	73
83	Direkte AST-Konstruktion in PLY Teil3	74
84	CST-zu-AST-Transformation in TypeScript	75
85	Rekursive Listen in der Python-Version	75
86	Verarbeitung rekursiver Listen in TypeScript	76
87	AST-Typdefinition in TypeScript	76
88	Ausführung von Anweisungen in Python	77
89	Ausführung von Anweisungen in TypeScript	78
90	Arithmetische Auswertung in Python	79
91	Arithmetische Auswertung in TypeScript Teil 1	80
92	Boolesche Auswertung in Python	81
93	Boolesche Auswertung in TypeScript	82
94	Programmausführung in Python	82
95	Programmausführung in TypeScript	83

1 Einleitung

Die Theoretische Informatik bildet ein wichtiges Fundament im Informatikstudium. Themen wie Automaten oder formale Sprachen können anfangs jedoch sehr abstrakt wirken. Um diese theoretischen Konzepte greifbarer zu machen, kommen in der Vorlesung „Theoretische Informatik III“ interaktive Notebooks zum Einsatz. Dort wird erklärender Text direkt mit ausführbarem Programmcode gemischt. Bislang basieren diese Notebooks auf der Programmiersprache Python. Studierende können so theoretische Algorithmen direkt ausprobieren, verändern und live nachvollziehen.

Diese Arbeit befasst sich mit der Übertragung dieser Lehrinhalte in die Programmiersprache TypeScript. Der Fokus liegt dabei auf einer Portierung, welche nicht nur die Syntax übersetzt, sondern die Konzepte der formalen Sprachen durch feste, verbindliche Datenstrukturen explizit modelliert.

1.1 Problemstellung

Ausgangspunkt dieser Arbeit sind interaktive Python-Notebooks, die Konzepte der theoretischen Informatik demonstrieren. Werden diese Lehrmaterialien in das TypeScript-Ökosystem übertragen, reicht es jedoch nicht aus, lediglich die Syntax der Sprache zu übersetzen. Es entstehen fundamentale technische Hürden, da beide Sprachen Daten auf unterschiedliche Weise verarbeiten. Folgende zwei Problemfelder stehen dabei im Fokus:

Zu späte Fehlererkennung bei verschachtelten Daten

Python ist eine dynamische Sprache. Das bedeutet, dass Variablen keine festen Typen haben und Fehler oft erst dann auffallen, wenn das Programm ausgeführt wird und abstürzt. Zwar gibt es Werkzeuge, um Typen in Python zu prüfen, in der interaktiven Umgebung eines Jupyter-Notebooks übersehen diese jedoch häufig Fehler.

In der Vorlesung arbeiten wir ständig mit tief verschachtelten Objekten – beispielsweise Datenstrukturen, bei denen eine Liste wieder viele weitere Objekte und Listen enthält. Fehlt einem dieser verschachtelten Elemente versehentlich eine wichtige Eigenschaft, merkt Python das erst, wenn der Code im Hintergrund genau dieses fehlerhafte Element aufruft. Wir benötigen stattdessen eine Sprache, die uns schon während des Tippens garantiert, dass unsere Datenstrukturen über alle Ebenen hinweg fehlerfrei und vollständig aufgebaut sind.

Das unerwartete Verhalten von Mengen (Sets)

Das gravierendste Hindernis bei der Übersetzung zeigt sich jedoch bei der Arbeit mit Mengen (Sets). In der theoretischen Informatik ist es unumgänglich, berechnete Zustände in Mengen zu speichern, um Duplikate zu vermeiden oder später zu prüfen, ob ein Zustand bereits erreicht wurde.

In Python funktioniert das exakt so, wie man es intuitiv erwartet. Fügen wir ein Tupel in eine Menge ein und fragen danach ab, ob ein Tupel mit denselben Zahlen enthalten ist, liefert Python das erwartete Ergebnis:

```
1  menge = set()
2  menge.add((1, 2))
3
4  # Die Abfrage liefert das erwartete Ergebnis
5  print((1, 2) in menge) # Ausgabe: True
```

Listing 1: Mengen in Python: Das erwartete Verhalten

Ein naiver Übersetzungsversuch nach TypeScript führt hier jedoch zu einem fundamentalen Bruch in der Logik. Nutzen wir die Standard-Datenstruktur Set in TypeScript auf die exakt gleiche Weise, erhalten wir plötzlich ein falsches Ergebnis:

```
1  const menge = new Set();
2  menge.add([1, 2]);
3
4  // TypeScript findet das Array überraschenderweise nicht
5  console.log(menge.has([1, 2])); // Ausgabe: false
```

Listing 2: Mengen in TypeScript: Das fehlerhafte Verhalten

Obwohl das Array `[1, 2]` im Code zuvor offensichtlich in die Menge eingefügt wurde, behauptet TypeScript, es sei nicht vorhanden. Dieser zunächst unerklärliche Unterschied führt bei einer direkten Portierung dazu, dass Logik-Abfragen fehlschlagen, Endlosschleifen entstehen und Algorithmen falsche Ergebnisse liefern. Die Ursachenforschung und Behebung dieses Verhaltens ist die zentrale technische Herausforderung dieser Arbeit.

1.2 Zielsetzung

Das übergeordnete Ziel dieser Arbeit ist es, die Lehrinhalte der Vorlesung „Theoretische Informatik III“ vollständig nach TypeScript zu übersetzen. Die neue Co-

debasis soll die bisherigen Python-Notebooks ersetzen und dabei die klaren Strukturvorgaben der neuen Sprache nutzen, um Fehler frühzeitig zu vermeiden.

Daraus leiten sich folgende konkrete Teilziele ab:

1. **Strukturierte Neuimplementierung:** Die Algorithmen der Vorlesung sollen in TypeScript portiert werden. Die dynamische, oft fehleranfälligere Natur von Python soll dabei durch verbindliche Datenstrukturen und Typvorgaben ersetzt werden.
2. **Lösung des Mengen-Problems:** Um die Algorithmen fehlerfrei auszuführen, muss das unerwartete Verhalten von TypeScripts Set analysiert und behoben werden. Ziel ist die Entwicklung einer maßgeschneiderten Datenstruktur, die sich exakt wie ein Python-Set verhält und verschachtelte Objekte verlässlich erkennt.
3. **Einheitliche Ausführungsumgebung:** Da die Notebooks lokal ausgeführt werden, muss eine standardisierte Umgebung geschaffen werden, die unabhängig vom Betriebssystem identisch funktioniert. Ein wichtiges technisches Teilziel ist es dabei, die Hintergrundprozesse der Notebooks so anzupassen, dass TypeScripts strenge Prüfregeln auch bei der interaktiven, schrittweisen Ausführung zuverlässig erzwungen werden. Der Einrichtungsaufwand für Studierende soll durch eine klare Schritt-für-Schritt-Anleitung minimal gehalten werden.

1.3 Methodik und Abgrenzung

Die methodische Durchführung dieser Arbeit folgt einem fokussierten Ansatz. Der Umfang der Portierung beschränkt sich auf die nummerierten Lehrnotebooks der Vorlesung sowie deren unmittelbare technische Abhängigkeiten. Übungsnotebooks, die primär dem unbetreuten Selbststudium dienen, sind explizit vom Umfang dieser Arbeit ausgeklammert. Die genaue Auswahl der zu übersetzenden Artefakte erfolgte in Abstimmung mit dem Betreuer und ist im Anhang (siehe Abschnitt A.1) tabellarisch aufgeführt.

Ein zentraler methodischer Aspekt betrifft die didaktische Aufbereitung: Die begleitenden Erklärtexpte der Notebooks wurden nicht isoliert übersetzt, sondern inhaltlich an die neuen Strukturen von TypeScript angepasst. Dabei wurde bewusst darauf verzichtet, die neuen Konzepte kontinuierlich mit der alten Python-Implementierung zu vergleichen. Die Notebooks sollen als eigenständiges Lehrmaterial funktionieren, das die theoretischen Konzepte nativ und in sich geschlossen vermittelt.

Auf technischer Ebene war die Minimierung von Systemabhängigkeiten ein wesentliches Entwurfsziel. Insbesondere für die grafische Visualisierung von Automaten und Graphen wurde darauf geachtet, keine komplexen lokalen Installationen von externer Software vorauszusetzen. Stattdessen kommen integrierbare, plattformunabhängige Lösungen zum Einsatz, um die Ausführung auf verschiedenen Betriebssystemen zu garantieren und den Einstieg für die Studierenden so barrierefrei wie möglich zu gestalten.

Zur Gewährleistung der Reproduzierbarkeit und für die freie Zugänglichkeit der Ergebnisse wird der gesamte entwickelte Quellcode in einem öffentlichen Repository bereitgestellt. Die Implementierung ist unter der URL <https://github.com/cstrerath/formal-languages-typescript> im Verzeichnis TypeScript abrufbar.

2 Theoretische Grundlagen der Typisierung

Beim Übergang von den interaktiven Python-Notebooks der Vorlesung zu einer TypeScript-Implementierung ändert sich nicht nur die Syntax. Der fundamentalste Unterschied liegt im Umgang mit Datentypen. Die in Python erprobten Algorithmen wurden ursprünglich in einer hochdynamischen Umgebung entwickelt und mittels „mypy“ typisiert. TypeScript verwendet einen anderen Ansatz, die Typisierung ist der zentrale Aspekt der Sprache, daher ist ebenjene in diesem Projekt kein Randthema, sondern das tragende Fundament der gesamten Codebasis.

2.1 Semantik und Funktion von Datentypen

Im Arbeitsspeicher eines Computers bestehen alle Daten letztlich nur aus Bitfolgen, also aus Nullen und Einsen. Für den Prozessor ist eine solche Bitfolge zunächst bedeutungslos. Erst durch einen „Typ“ erhält sie eine fachliche Interpretation. Der Typ legt fest, ob ein Wert beispielsweise als Zahl, als Text, als Wahrheitswert oder als komplexe Datenstruktur behandelt wird.

Man kann einen Typ daher als eine Art Vertrag verstehen. Dieser Vertrag beschreibt die Form eines Wertes und definiert, welche Operationen auf ihm sinnvoll und erlaubt sind. Ein kurzes Python-Beispiel (siehe Listing 3) verdeutlicht dieses Prinzip: Eine Zahl mit einer anderen Zahl zu summieren ist logisch, ebenso die Verkettung zweier Strings. Die Addition von String und Zahl führt hingegen zu einem inhaltlichen Konflikt.

```
1 zahl = 42
2 text = "42"
3 print(zahl + 1)      # sinnvoll: 43
4 print(text + "!")    # sinnvoll: "42!"
5 print(text + 1)      # nicht sinnvoll: Text und Zahl passen hier nicht zusammen
```

Listing 3: Unterschiedliche Operationen erfordern unterschiedliche Typen

Ob eine Operation zulässig ist, hängt demnach maßgeblich vom Typ ab. Genau hier setzen Typsysteme an: Sie strukturieren die Bedeutung von Daten und dienen als essenzieller Sicherheitsmechanismus. Wenn ein Programm versucht, eine Operation auf einem ungeeigneten Wert auszuführen, liegt oft kein syntaktischer Fehler vor, sondern ein inhaltlicher Widerspruch. Ein Typsystem macht solche Widersprüche sichtbar, bevor sie zu schwer nachvollziehbaren Abstürzen im laufenden Programm führen.

Prinzipien der dynamischen Typisierung

Python und JavaScript sind dynamisch typisierte Sprachen. Das bedeutet, dass Variablen nicht dauerhaft an einen vorher festgelegten Typ gebunden sind. Der Typ ergibt sich stattdessen aus dem Wert, der zur Laufzeit gerade in der Variablen gespeichert ist. Wie Listing 4 zeigt, kann eine Variable in Python nahtlos nacheinander völlig unterschiedliche Datentypen aufnehmen.

```
1 value = 42
2 value = "Hallo"
3 value = [1, 2, 3]
4 print(value)
```

Listing 4: Dynamische Typisierung in Python

Dasselbe Grundprinzip findet sich unverändert in JavaScript wieder (siehe Listing 5), wenn man das Keyword `let` verwendet.

```
1 let value = 42;
2 value = "Hallo";
3 value = [1, 2, 3];
4 console.log(value);
```

Listing 5: Dynamische Typisierung in JavaScript

Diese Flexibilität erleichtert das schnelle Schreiben von Programmcode. Gleichzeitig verschiebt sie einen großen Teil der Verantwortung auf die Entwickelnden, da sich oft erst im Moment der Ausführung zeigt, ob eine Operation zum aktuellen Wert passt. Die möglichen Folgen dieser späten Auswertung demonstriert Listing 6.

```
1 value = [1, 2, 3]
2 print(value + 1)
```

Listing 6: Typfehler wird erst zur Laufzeit sichtbar

Obwohl der inhaltliche Fehler, die Addition von Liste und Ganzzahl, im Quelltext sofort ersichtlich ist, bemerkt Python ihn erst bei der Ausführung. In komplexen Algorithmen erschwert dies die Fehlererkennung erheblich.

2.2 Statische Typisierung mittels TypeScript

TypeScript ist keine völlig eigenständige Sprache, sondern eine statische Erweiterung von JavaScript. Während JavaScript die Laufzeitgrundlage bildet, fügt TypeScript Typinformationen hinzu, die bereits vor der Ausführung vom Compiler geprüft werden.

Zur Veranschaulichung zeigt Listing 7 zunächst eine einfache Addierfunktion in purem JavaScript.

```
1 function add(a, b) {  
2     return a + b;  
3 }  
4 console.log(add(3, 4));  
5 console.log(add("Hallo", "Welt"));
```

Listing 7: Eine untypisierte Funktion in JavaScript

Da keine Typen vorgegeben sind, akzeptiert die Funktion beliebige Eingaben. Die Flexibilität ist hoch, die eigentliche fachliche Absicht der Funktion bleibt jedoch im Dunkeln. Listing 8 zeigt im direkten Kontrast, wie TypeScript es erlaubt, diese Absicht unmissverständlich im Code festzuhalten.

```
1 function add(a: number, b: number): number {  
2     return a + b;  
3 }  
4 console.log(add(3, 4));  
5 // console.log(add("Hallo", "Welt")); // Führt zu einem Compiler-Fehler
```

Listing 8: Dieselbe Funktion in TypeScript mit expliziten Typen

Durch die expliziten Typannotationen wird die Signatur, also die formale Schnittstelle bestehend aus Funktionsnamen sowie Anzahl, Reihenfolge und Datentypen der Parameter, eindeutig. Ein fehlerhafter Aufruf mit zwei Strings widerspricht diesem Vertrag und wird vom Compiler beanstandet, lange bevor das Programm ausgeführt wird. Der hier verwendete Typ `number` ist dabei der zentrale Baustein für numerische Werte:

Definition 2.1. Der Typ `number`: Im Gegensatz zu Sprachen wie Python, C oder Java, die streng zwischen ganzen Zahlen (`int`) und Kommazahlen (`float`) unterscheiden, fasst TypeScript alle numerischen Werte unter dem Typ `number` zusammen. Programmatisch handelt es sich um 64-Bit-Fließkommazahlen nach dem IEEE-754-Standard [1]. Eine Funktion mit diesem Parameter akzeptiert folglich sowohl `add(3, 4)` als auch `add(3.14, 2.5)`.

2.2.1 Herausforderungen einer naiven Code-Migration

Eine reine Portierung von Python-Code nach TypeScript führt nicht automatisch zu echter Typsicherheit. Da TypeScript als Obermenge von JavaScript konzipiert ist, lässt sich bestehender Python-Code häufig fast unverändert (lediglich mit geänderter Syntax) übernehmen, bleibt im Kern jedoch reines JavaScript. Listing 9 demonstriert eine solche naive Übersetzung.

```
1 function formatUserId(id) {  
2     return "User_" + id;  
3 }  
4 console.log(formatUserId(5));  
5 console.log(formatUserId("admin"));  
6 console.log(formatUserId([1, 2, 3]));
```

Listing 9: Naive Übersetzung einer Funktion nach TypeScript

Die Funktion kompiliert ohne Fehlermeldung, verarbeitet aber völlig unterschiedslos Zahlen, Strings und Arrays. Die Absicht, hier die Formatierung einer numerischen Nutzer-ID, geht komplett verloren. Ein solcher Code ist zwar formal TypeScript, inhaltlich bewahrt er jedoch die Fehleranfälligkeit der dynamischen Programmierung.

Dynamische Typprüfung zur Laufzeit

Auch in untypisierten Umgebungen besitzen Daten zur Laufzeit stets einen konkreten Typ, der sich programmatisch auslesen lässt. In Python wird hierfür typischerweise die Funktion `type()` verwendet (siehe Listing 10), während JavaScript und TypeScript den Operator `typeof` nutzen (siehe Listing 11).

Implikationen fehlender Typisierung (`any`)

Werden Variablen in TypeScript nicht explizit annotiert und lässt sich ihr Typ nicht eindeutig ableiten (wie bei dem Parameter `id` in der naiven Übersetzung), greift TypeScript auf einen Fallback zurück: Es weist der Variable den Typ `any` zu. Dieser Typ nimmt eine gefährliche Sonderrolle im System ein.

```

1 print(type(42))           # <class 'int'>
2 print(type("Hallo"))     # <class 'str'>
3 print(type([1, 2, 3]))    # <class 'list'>

```

Listing 10: Typprüfung zur Laufzeit in Python

```

1 console.log(typeof 42);    // "number"
2 console.log(typeof "Hallo"); // "string"
3 console.log(typeof true);  // "boolean"

```

Listing 11: Typprüfung zur Laufzeit in JavaScript und TypeScript

Definition 2.2. Der Typ `any`: Das Schlüsselwort `any` ist der universelle Fluchtweg aus dem Typsystem. Es repräsentiert einen beliebigen JavaScript-Wert ohne jegliche Einschränkungen. Der Compiler erlaubt auf einem `any`-Wert den Aufruf beliebiger Methoden und Eigenschaften, womit die statische Typprüfung für diese Variable faktisch vollständig deaktiviert wird.

Wie Listing 12 verdeutlicht, kann `any` auch explizit vergeben werden, um den Compiler bewusst „blind“ zu machen.

```

1 let value: any = [1, 2, 3];
2 console.log(value.toUpperCase()); // stürzt zur Laufzeit ab
3 console.log(value * 2);           // ergibt NaN
4 console.log(value.nonExistingMethod()); // stürzt zur Laufzeit ab

```

Listing 12: Der Typ `any` hebt die Typprüfung aus

Ein typsicheres Programm müsste validieren, ob `value` überhaupt die Methode `toUpperCase()` besitzt. Bei `any` entfällt diese Kontrolle, was zu Laufzeitfehlern führen kann.

Gefahren der impliziten Typkonvertierung (Koerzision)

Ein weiteres Risiko schlecht typisierten TypeScript-Codes ist die sogenannte *Koerzision* (implizite Typumwandlung). Python verlangt für fast alle Operationen explizite Umwandlungen, wie Listing 13 zeigt.

JavaScript (und untypisiertes TypeScript) ist an dieser Stelle nachgiebiger und erzwingt oft stillschweigende Umwandlungen, sobald ein Operator dies nahelegt. Listing 14 stellt dieses Verhalten der expliziten Umwandlung gegenüber.

```

1 values = [1, 2, 3]
2 text = str(values)
3 print(text)          # "[1, 2, 3]"

```

Listing 13: Explizite Umwandlung in Python

```

1 const values = [1, 2, 3];
2 const implicitText = values + "";    // Koerzision
3 const explicitText = String(values); // Explizite Umwandlung
4 console.log(implicitText);           // "1,2,3"

```

Listing 14: Implizite und explizite Umwandlung in JavaScript

Type Assertions als trügerische Compiler-Anweisung

Neben `any` bietet TypeScript mit der Typbehauptung einen weiteren Mechanismus, der das Typsystem aufweicht und für Einsteiger oft wie eine bequeme Typumwandlung (Type Cast) wirkt. Dies ist jedoch ein Trugschluss.

Definition 2.3. Die Type Assertion (as): Das Schlüsselwort `as` ist keine Operation, die zur Laufzeit ausgeführt wird. Es ist eine reine Compiler-Direktive. Sie zwingt den TypeScript-Compiler dazu, einen Wert ab sofort als den angegebenen Typ zu behandeln, lässt den Wert im Arbeitsspeicher aber völlig unangetastet.

Listing 15 verdeutlicht diese trügerische Sicherheit.

```

1 let values: any = [1, 2, 3];
2
3 // Keine echte Laufzeit-Umwandlung, sondern nur eine Behauptung:
4 const text = values as string;
5
6 // Der Wert ist zur Laufzeit weiterhin ein Array!
7 console.log(text.toUpperCase()); // Laufzeitfehler

```

Listing 15: Eine Type Assertion ist keine echte Umwandlung

Das inhaltliche Problem wurde somit nicht gelöst, sondern lediglich vor der Typprüfung versteckt, zur Laufzeit ist die Methode auf dem Array nicht definiert und es kommt zum Absturz.

Methodischer Verzicht auf Typ-Bypässe

Weder `any` noch `as` sind neutrale Komfortfunktionen. Es sind Mechanismen, die die Wirkung des Typsystems gezielt aushebeln. Das erklärte Ziel dieser Studienarbeit war es, den Code nach TypeScript, und nicht bloß nach JavaScript in `.ts`-Dateien, zu überführen. Folglich wurde architektonisch in der gesamten Implementierung auf den Einsatz von `any` und ungesicherten `as`-Behauptungen verzichtet.

2.2.2 Explizite Typannotationen

Der professionelle Gegenentwurf zu Ausweichmechanismen ist die explizite Typannotation. Dabei wird im Programmtext unmissverständlich angegeben, welcher Typ für Variablen, Parameter oder Rückgabewerte zwingend erwartet wird.

Listing 16 zeigt den Unterschied anhand einer Funktion, die aus einer numerischen ID einen formatierten Text generiert.

```
1 // Unpräzise: Lässt völlig unklar, was id sein soll
2 function formatUserIdLoose(id) {
3     return "User_" + id;
4 }
5
6 // Präzise: Zwingender Vertrag durch explizite Annotationen
7 function formatUserIdStrict(id: number): string {
8     return "User_" + id;
9 }
```

Listing 16: Unpräzise vs. präzise formulierte Funktion in TypeScript

Die strikte Variante formuliert einen eindeutigen Vertrag, der keinen Interpretationsspielraum zulässt. Die Signatur verlangt nicht nur eine `number` als Eingabe, sondern garantiert auch einen spezifischen Rückgabety, in diesem Fall einen Text:

Definition 2.4. Der Typ `string`: Dieser primitive Typ repräsentiert jegliche Form von Textdaten. Er besteht aus einer unveränderlichen (immutablen) Sequenz von 16-Bit-Werten nach dem UTF-16-Standard. Einmal im Speicher angelegt, kann ein String nicht modifiziert werden; Operationen wie String-Verkettungen erzeugen stets eine neue Zeichenkette.

Atomare und strukturierte Basistypen

Neben den bereits eingeführten primitiven Werten wie Zahlen und Texten, ist ein dritter atomarer Baustein essenziell für die Steuerung des Programmflusses:

Definition 2.5. Der Typ `boolean`: Er repräsentiert logische Wahrheitswerte und kann ausschließlich die Zustände `true` oder `false` annehmen. Er entspricht exakt dem `bool`-Typ aus Python und ist essenziell für Kontrollfluss-Bedingungen.

Zusätzlich zu diesen atomaren Bausteinen erlaubt TypeScript die Definition zusammengesetzter Strukturen. Wie Listing 17 demonstriert, können Arrays oder spezifisch geformte Objekte direkt annotiert werden.

```
1 // Atomare Typen
2 let username: string = "alice";
3 let loginCount: number = 5;
4 let isActive: boolean = true;
5
6 // Zusammengesetzte Typen
7 let roles: string[] = ["admin", "staff"];
8 let userProfile: { id: number; name: string } = {
9     id: 42,
10    name: "Alice"
11 };
```

Listing 17: Atomare und zusammengesetzte TypeScript-Typen

Bereits die simple Typdefinition von `userProfile` legt eine klare Struktur mit zwei benannten und typisierten Bestandteilen fest. Die Fähigkeit, komplexe Verträge im Code zu verankern, ist Grundlage sicherer Architekturen.

Eingrenzung von Wertebereichen durch Literal und Union Types

Oft reicht es nicht aus, einen Wert pauschal als `string` zu deklarieren. Wenn fachlich nur eine feste Menge spezifischer Ausprägungen zulässig ist (etwa bestimmte Nutzerrollen), bietet TypeScript zwei eng verwobene Konzepte:

Definition 2.6. Literal Types: Ein Literal Type beschreibt nicht eine ganze Menge von Werten, sondern repräsentiert exakt einen einzigen, konkreten Wert (z. B. exakt die Zeichenkette `"admin"`).

Definition 2.7. Union Types (|): Ein Union Type kombiniert mehrere Typen durch ein logisches „Oder“. Eine Variable dieses Typs darf zur Laufzeit jeden der verknüpften Typen annehmen.

Ihre immense Flexibilität gewinnen Literal Types gerade durch diese Kombination mit Union Types. Listing 18 zeigt, wie sich Zustandsräume dadurch präzise eingrenzen lassen.

```
1 type Role = "admin" | "user" | "guest";
2
3 let currentRole: Role = "user";
4 currentRole = "admin";
5 // currentRole = "superuser";
6 // Compiler-Fehler: Typ '"superuser"' ist nicht zuweisbar
```

Listing 18: Union Type aus Literal Types in TypeScript

Dieses Konzept transformiert lose Text-Konventionen in maschinenprüfbare Verträge. Python unterstützt dies ebenfalls sehr ähnlich und nutzt dafür `Literal` aus dem `typing`-Modul, wie Listing 19 belegt.

```
1 from typing import Literal
2
3 Role = Literal["admin", "user", "guest"]
4
5 def is_privileged(role: Role) -> bool:
6     return role == "admin"
```

Listing 19: Literal Types in Python

Modellierung fehlender Werte (null und undefined)

In realen Programmen tritt häufig der Fall auf, dass ein gesuchter Wert schlicht nicht vorhanden ist. Ein präzises Typsystem muss die Möglichkeit eines solchen fehlenden Wertes ausdrücklich modellieren, anstatt einfach darauf zu hoffen, dass der aufrufende Code damit zurechtkommt. TypeScript verwendet dafür zwei spezifische Typen, die verschiedene Nuancen der Abwesenheit ausdrücken:

Definition 2.8. Die Typen `undefined` und `null`: Der Typ `undefined` bedeutet typischerweise, dass ein Wert noch nicht initialisiert wurde oder in einem Suchvorgang nicht gefunden werden konnte. Der Typ `null` wird hingegen meist genutzt, um die absichtliche, bewusste Abwesenheit eines Wertes durch den Entwickler zu markieren. Beide Typen repräsentieren zur Laufzeit einen fehlenden Wert, haben aber eine leicht unterschiedliche Semantik.

Die Suche nach einer E-Mail-Adresse in einer Datenbank, wie in Listing 20 dargestellt, verdeutlicht dieses Prinzip.

```
1 function findEmail(username: string): string | undefined {  
2     if (username === "admin") {  
3         return "admin@example.com";  
4     }  
5     return undefined;  
6 }
```

Listing 20: Ein möglicher fehlender Rückgabewert in TypeScript

Da die Funktion nicht zwingend ein Ergebnis findet, reicht der Rückgabotyp `string` allein nicht aus. Die Signatur `string | undefined` nutzt einen Union Type und hält somit ausdrücklich fest, dass entweder ein String oder eben kein nutzbarer Wert zurückkommt.

Python modelliert dieses Problem auf der Ebene der Laufzeitwerte ganz ähnlich, wie Listing 21 zeigt. Dort wird ein potenziell fehlender Wert typischerweise durch `None` ausgedrückt.

```
1 def find_email(username):  
2     if username == "admin":  
3         return "admin@example.com"  
4     return None
```

Listing 21: Möglicher fehlender Rückgabewert in Python

Der Grundgedanke ist in beiden Sprachen identisch: Ein Rückgabewert kann vorhanden sein oder fehlen. Entscheidend ist jedoch, wie streng diese Möglichkeit anschließend durch den Compiler der jeweiligen Sprache eingefordert wird.

Strikte Nullwert-Prüfung (`strictNullChecks`)

Die bloße Existenz von `null` und `undefined` erzeugt noch keine automatische Typsicherheit. Ob der Compiler diese Fälle als eigenständige Problemquellen ernst nimmt, steuert in TypeScript eine spezifische Konfigurationseinstellung:

Definition 2.9. Die Compiler-Option `strictNullChecks`: Ist diese Option deaktiviert, behandelt TypeScript fehlende Werte nachsichtig, `null` und `undefined` dürfen überall dort übergeben werden, wo eigentlich beispielsweise ein `string` erwartet wird. Ist sie jedoch aktiviert, trennt der Compiler diese Typen strikt von regulären Werten. Ein potenziell fehlender Wert muss dann zwingend im Code auf seine Existenz geprüft werden, bevor auf ihn zugegriffen werden darf.

Führt man das vorherige E-Mail-Beispiel fort, wird die Tragweite dieser Option in Listing 22 sofort sichtbar.

```
1 const email = findEmail("guest");
2 console.log(email.toUpperCase()); // Compiler-Fehler bei aktivem strictNullChecks
```

Listing 22: Fehlerhafte Nutzung ohne vorherige Prüfung

Diese Zeile ist hochgradig fehleranfällig. Liefert die Suche `undefined`, schlägt der Aufruf von `toUpperCase()` zur Laufzeit fehl. Mit aktiviertem `strictNullChecks` macht der TypeScript-Compiler diesen Widerspruch schon während des Tippens im Editor sichtbar und verweigert die Übersetzung.

Die saubere Lösung besteht darin, den fehlenden Wert im Kontrollfluss explizit zu behandeln, wie Listing 23 demonstriert.

```
1 const email = findEmail("guest");
2 if (email !== undefined) {
3     console.log(email.toUpperCase()); // email kann nur string sein
4 }
```

Listing 23: Explizite Behandlung eines möglichen `undefined`-Werts

Einer der häufigsten Fehler in Programmen besteht darin, blind davon auszugehen, dass ein Wert vorhanden ist. Für die Notebooks war `strictNullChecks` deshalb keine kosmetische Zusatzregel, sondern eine elementare Sicherheitsmaßnahme, welche über einen Patch am Notebookkernel eingebaut wurde.

Der Typ `unknown` als sichere Abstraktion

Neben `null` und `undefined` bietet TypeScript einen weiteren wichtigen Spezialfall an, um mit Unsicherheit umzugehen. Dieser ist besonders dann nützlich, wenn zwar feststeht, dass ein Wert existiert, dessen genauer Typ aber noch nicht bekannt ist (beispielsweise bei externen Eingaben).

Definition 2.10. Der Typ `unknown`: Dieser Typ ist das typsichere Gegenstück zu `any`. Während `any` die Typprüfung faktisch deaktiviert, verbietet `unknown` zunächst jegliche Interaktion. Der Compiler zwingt den Entwickler dazu, den Wert durch Laufzeitprüfungen präzise einzuengen, bevor typspezifische Methoden aufgerufen werden dürfen.

Listing 24 zeigt, wie der Compiler unsichere Operationen auf einem `unknown`-Wert rigoros blockiert.

```
1 let value: unknown = [1, 2, 3];
2
3 // Nicht erlaubt, solange der Typ noch unbekannt ist:
4 // console.log(value.toUpperCase()); // Compiler-Fehler
```

Listing 24: Der Typ `unknown` erzwingt eine vorherige Prüfung

Erst nach einer erfolgreichen Typprüfung im Kontrollfluss schaltet der Compiler die typspezifischen Methoden frei, was in Listing 25 ersichtlich wird.

```
1 let value: unknown = "Alice";
2
3 if (typeof value === "string") {
4     // Innerhalb dieses Blocks weiß der Compiler, dass value ein string ist.
5     console.log(value.toUpperCase());
6 }
```

Listing 25: Einengung von `unknown` durch Typprüfung im Kontrollfluss

Auf diese Weise lassen sich unsichere Daten sicher modellieren, ohne den Type Checker auszuhebeln.

Vergleich mit statischer Analyse in Python (mypy)

Auf den ersten Blick besitzen Python und TypeScript ähnliche Ausdrucksmittel für Typen. Wie Listing 26 zeigt, lassen sich auch in Python Variablen, Parameter und Rückgabewerte mittels Type Hints annotieren.

```
1 def format_user_id(user_id: int) -> str:
2     return f"User_{user_id}"
```

Listing 26: Type Hints in Python

Ein fundamentaler Unterschied besteht jedoch in der Durchsetzung: Python ignoriert diese Type Hints zur Laufzeit vollständig. Ein fehlerhafter Aufruf mit falschen Datentypen führt nicht zwingend zu einem sofortigen Fehler. Um die Typinformationen systematisch zu prüfen, wird ein externes Werkzeug wie *mypy* benötigt, das den Code statisch analysiert.

Zudem fehlt Python ein strenges Äquivalent zum sicheren `unknown`. Der Typ `Any` in Python entspricht vielmehr dem unsicheren `any` in TypeScript. Listing 27 stellt diese Konzepte direkt gegenüber.

```
1 from typing import Any
2
3 def normalize(value: Any) -> str:
4     return str(value) # mypy erlaubt hier völlig beliebige Eingaben
```

```
1 function normalize(value: unknown): string {
2     if (typeof value === "string") {
3         return value;
4     }
5     return String(value); // TypeScript erzwingt die Prüfung oder Konvertierung
6 }
```

Listing 27: Python Any vs. TypeScript unknown

2.2.3 Strukturelles Typing und Interface-Verträge

Neben primitiven Werten erfordert die Modellierung von Software verlässliche Verträge zwischen Komponenten. Ein solcher Vertrag legt fest, welche Eigenschaften oder Methoden ein Wert besitzen muss, um genutzt werden zu können. TypeScript verwendet dafür unter anderem *Interfaces*.

Entscheidend ist dabei ein Konzept, das TypeScript grundlegend von Sprachen wie Java oder C# unterscheidet:

Definition 2.11. Strukturelles Typing (Statisches Duck-Typing): TypeScript prüft Typkompatibilität nicht anhand einer expliziten Klassenhierarchie (Nominales Typing), sondern rein anhand der „Gestalt“ (Struktur) eines Objekts. Besitzt ein Objekt alle Eigenschaften und Methoden, die ein Interface verlangt, gilt es als kompatibel zu diesem Interface, unabhängig davon, ob es explizit davon erbt oder nicht.

Listing 28 definiert einen solchen strukturellen Vertrag in Form des Interfaces `Displayable`.

```

1 interface Displayable {
2     getLabel(): string;
3     isActive(): boolean;
4 }

```

Listing 28: Ein Interface als Vertrag über benötigte Methoden

Dieses Interface fordert lediglich das Vorhandensein zweier spezifischer Methoden. Listing 29 demonstriert, dass jede Klasse, die diese Signatur zufällig oder absichtlich erfüllt, automatisch typkompatibel ist.

```

1 class UserAccount {
2     constructor(public username: string, public active: boolean) {}
3
4     getLabel(): string {
5         return this.username;
6     }
7
8     isActive(): boolean {
9         return this.active;
10    }
11 }
12
13 // Zuweisung ist zulässig, da Struktur des Objekts mit Interface übereinstimmt:
14 const item: Displayable = new UserAccount("alice", true);

```

Listing 29: Eine Klasse erfüllt ein Interface über ihre reine Struktur

Dieser strukturelle Ansatz entkoppelt Funktionalität von starren Klassenhierarchien. Datenstrukturen können polymorph mit völlig unterschiedlichen Elementen arbeiten, solange diese den vertraglich zugesicherten Methodenapparat bereitstellen.

Äquivalente Strukturierung in Python (Protocol und TypedDict)

Python unterstützt strukturelles Denken zur Laufzeit nativ durch sein dynamisches Duck-Typing. Um dieses Verhalten jedoch statisch überprüfbar zu machen, bietet das `typing`-Modul (in Kombination mit `mypy`) spezifische Hilfsmittel an.

Analog zu einem TypeScript-Interface ermöglicht die Klasse `Protocol` die Definition von Methoden-Verträgen. Listing 30 zeigt, wie ein solcher statischer Vertrag in Python formuliert wird.

Klassen müssen auch in Python nicht explizit von diesem Protocol erben, um als kompatibel zu gelten; die bloße Erfüllung der Signatur reicht aus. Während

```

1 from typing import Protocol
2
3 class Displayable(Protocol):
4     def get_label(self) -> str: ...
5     def is_active(self) -> bool: ...

```

Listing 30: Ein Protocol in Python als statischer Vertrag

strukturelle Kompatibilität in TypeScript jedoch das fundamentale Designprinzip bildet, bleibt sie in Python ein optionales Konstrukt.

Für reine Datenobjekte (ohne eigenes Verhalten) bietet Python zusätzlich `TypedDict`. Wie Listing 31 demonstriert, lassen sich damit Dictionary-Strukturen mit festen Schlüsseln und Typen definieren.

```

1 from typing import TypedDict
2
3 class Address(TypedDict):
4     street: str
5     zip_code: int
6     city: str

```

Listing 31: Feste Datenstruktur mit TypedDict in Python

Interfaces vs. Type Aliase in TypeScript

In TypeScript gibt es für die Modellierung von Datenstrukturen gleich zwei Werkzeuge, die auf den ersten Blick identisch wirken: `interface` und `type`. Während `interface` (wie in Abschnitt 2.2.3 gezeigt) historisch aus der objektorientierten Welt stammt und primär die Gestalt von Objekten oder Klassen beschreibt, ist `type` ein deutlich universelleres Konzept:

Definition 2.12. Type Alias (`type`): Ein Type Alias gibt einem beliebigen Typ einen neuen, aussagekräftigen Namen. Im Gegensatz zu einem `interface` kann ein `type` nicht nur Objektstrukturen, sondern auch die bereits eingeführten Union Types, Tupel oder primitive Werte benennen.

Listing 32 stellt die TypeScript-Äquivalenz zum Python- `TypedDict` dar. Es zeigt, wie fließend `interface` und `type` für reine Datenobjekte ineinander übergehen, während `type` bei komplexeren Kombinationen (wie Union Types) alternativlos ist.

Für die Architektur der portierten Software bedeutet dies eine klare Arbeitsteilung: Sobald Verträge für Klassen oder polymorphes Verhalten (Methoden)

```

1 // Variante 1: Interface (Fokus auf die Objektform, oft nachträglich erweiterbar)
2 interface Address {
3     street: string;
4     zipCode: number;
5     city: string;
6 }
7
8 // Variante 2: Type Alias (Fokus auf einen festen Namen für eine feste Struktur)
9 type UserData = {
10     name: string;
11     email: string;
12     address: Address; // Nahtlose Kombination von Type und Interface
13 };
14
15 // Die exklusive Stärke von 'type': Benennung von Unions oder Tupeln
16 // (Dies wäre mit einem 'interface' syntaktisch nicht möglich)
17 type UnsafeMail = string | undefined;

```

Listing 32: Objektstrukturen und Aliase: interface und type in TypeScript

definiert werden, greift die Codebasis bevorzugt auf `interface` zurück. Geht es jedoch um die Definition von reinen Datencontainern, Tupeln oder die Kombination von Typen (wie bei `UnsafeMail`), ist der `type`-Alias das Werkzeug der Wahl. Diese saubere semantische Trennung verbessert die Lesbarkeit und macht die TypeScript-Notebooks in ihrer Typbeschreibung ausdrucksstärker als die Python-Vorlage.

Generische Programmierung im Sprachvergleich

Um Algorithmen und Datenstrukturen typsicher, aber dennoch hochgradig wiederverwendbar zu gestalten, bedarf es einer weiteren Abstraktionsebene:

Definition 2.13. Generische Programmierung (Generics): Generics fungieren als Variablen für Typen. Anstatt eine Funktion starr auf einen Typen festzulegen oder auf das unsichere `any` auszuweichen, wird ein Typ-Parameter (meist `T`) definiert. Dieser Platzhalter wird erst zum Zeitpunkt des tatsächlichen Aufrufs durch den konkreten Datentyp ersetzt.

Listing 33 demonstriert den Einsatz einer solchen generischen Funktion in TypeScript.

Dasselbe Konzept wird in Python mittels `TypeVar` umgesetzt, wie der direkte Vergleich in Listing 34 zeigt.

Die wahre architektonische Stärke von Generics entfaltet sich in der Kombinati-

```

1 function first<T>(items: T[]): T | undefined {
2     if (items.length === 0) return undefined;
3     return items[0];
4 }
5
6 // firstNumber ist number | undefined
7 const firstNumber = first<number>([10, 20, 30]);
8 // firstName ist string | undefined
9 const firstName = first<string>(["Alice", "Bob"]);

```

Listing 33: Eine generische Funktion in TypeScript

```

1 from typing import TypeVar
2
3 T = TypeVar("T")
4
5 def first(items: list[T]) -> T | None:
6     if len(items) == 0:
7         return None
8     return items[0]

```

Listing 34: Eine generische Funktion in Python

on mit strukturellen Schranken (Type Constraints). Listing 35 veranschaulicht, wie sich mit dem Schlüsselwort `extends` (in TypeScript) oder dem Parameter `bound` (in Python) erzwingen lässt, dass der variabel eingesetzte Typ einen bestimmten Basis-Vertrag erfüllen muss.

Kontrollflussbasierte Typverfeinerung (Type Narrowing)

Sobald ein Wert als Union Type definiert ist, verlangt der Compiler Klarheit darüber, welcher Fall zur Laufzeit konkret vorliegt, bevor typspezifische Operationen zugelassen werden.

Definition 2.14. Type Narrowing: Dies ist der Prozess der schrittweisen Typ-Einengung. Der TypeScript-Compiler analysiert den Kontrollfluss des Codes (wie `if`-Verzweigungen) in Kombination mit Laufzeitprüfungen. Kann bewiesen werden, dass eine Variable einen bestimmten Typ hat, behandelt der Compiler sie innerhalb dieses Blocks vollautomatisch als diesen engeren Typ.

Listing 36 verdeutlicht, wie diese statische Analyse in der Praxis auf Basis des `typeof`-Operators funktioniert.

Während `typeof` für primitive Datentypen geeignet ist, liefert es bei Instanzen

```

1 // TypeScript: T muss mindestens die Methoden aus 'Displayable' besitzen
2 function filterActive<T extends Displayable>(items: T[]): T[] {
3     return items.filter(item => item.isActive());
4 }

```

```

1 # Python: S wird an das Protocol 'Displayable' gebunden
2 S = TypeVar("S", bound=Displayable)
3
4 def filter_active(items: list[S]) -> list[S]:
5     return [item for item in items if item.is_active()]

```

Listing 35: Generics mit strukturellen Schranken im Vergleich

```

1 function formatValue(value: string | number): string {
2     if (typeof value === "string") {
3         // Narrowing greift: value ist hier zwingend ein string
4         return value.toUpperCase();
5     }
6     // Durch den frühen Return oben MUSS value hier eine number sein
7     return value.toFixed(2);
8 }

```

Listing 36: Type Narrowing durch Kontrollfluss und typeof

komplexer Klassen pauschal nur `"object"`. Für die Überprüfung von Objekten bietet sich daher der Operator `instanceof` an. Listing 37 demonstriert diese klassenbasierte Verfeinerung.

```
1 class UserAccount {
2     constructor(public username: string, public active: boolean) {}
3 }
4
5 function describe(user: UserAccount | string): string {
6     if (user instanceof UserAccount) {
7         return `Konto: ${user.username} (Aktiv: ${user.active})`;
8     }
9     return `Gast: ${user}`;
10 }
```

Listing 37: Klassenprüfung mit `instanceof`

Vollständigkeitsprüfung mittels `never`-Typ

Das Type Narrowing wirft unweigerlich eine logische Folgefrage auf: Welchen Typ hat ein Wert, wenn alle theoretisch möglichen Fälle einer Union vollständig abgehandelt wurden? In TypeScript nimmt dieser logisch un erreichbare Zustand einen besonderen Typ an:

Definition 2.15. Der Typ `never`: Dieser Typ repräsentiert einen Wert, der niemals existieren darf und Code, der niemals erreicht werden sollte. Er bildet den tiefsten Punkt in der Typhierarchie. Nichts ist `never` zuweisbar.

`never` wird in der Praxis als genialer Trick für das sogenannte *Exhaustiveness Checking* (Vollständigkeitsprüfung) eingesetzt. Listing 38 zeigt, wie dieser Typ garantiert, dass Erweiterungen des Datenmodells (z. B. ein neuer Bestellstatus im Online-Shop) vom Entwickler niemals unbemerkt übersehen werden können.

2.3 Fazit zur architektonischen Neuausrichtung

Die Herausforderung der ursprünglichen Architektur lag nicht in einer grundsätzlichen Unzulänglichkeit von *mypy* als Werkzeug. Für klassische, dateibasierte Python-Projekte ist *mypy* ein leistungsfähiger statischer Checker. Die Schwierigkeiten resultierten vielmehr aus der Diskrepanz zwischen einer nachträglich aufgesetzten statischen Typprüfung und der hochdynamischen, zellenbasierten Laufzeitumgebung von Jupyter-Notebooks.

```

1 type OrderStatus = "pending" | "shipped" | "delivered";
2
3 function translateStatus(status: OrderStatus): string {
4     switch (status) {
5         case "pending": return "In Bearbeitung";
6         case "shipped": return "Versendet";
7         case "delivered": return "Zugestellt";
8
9         default: {
10             // Compile-Fehler, falls OrderStatus in Zukunft z.B. um "canceled"
11             // erweitert wird, ohne diesen Switch-Block anzupassen:
12             const exhaustiveCheck: never = status;
13             return exhaustiveCheck;
14         }
15     }
16 }

```

Listing 38: Vollständige Fallunterscheidung mit `never`

In der Python-Umgebung musste die Typprüfung durch externe Werkzeuge wie *nb-mypy* nachgerüstet werden. Im praktischen Lehrbetrieb offenbarten sich dabei zwei gravierende architektonische Reibungspunkte:

- **Fehlendes Type Narrowing bei `match-case`:** Bei der Verwendung moderner Sprachkonstrukte wie dem strukturellen Pattern Matching (`match` und `case`) stieß die Notebook-Integration des Checkers an ihre Grenzen. Selbst wenn alle theoretisch möglichen Fälle logisch vollständig abgehandelt waren, erkannte das Tool diese Vollständigkeit (Exhaustiveness) nicht zuverlässig. Um Fehlermeldungen über angeblich fehlende Rückgabewerte zu vermeiden, mussten gezwungenermaßen toter Code geschrieben (z. B. ein redundantes `return None`) oder legitime Konstrukte per `# type: ignore` unterdrückt werden.
- **Zustandsverlust bei `%run`-Befehlen:** Um Code aus Basis-Notebooks in anderen Notebooks wiederzuverwenden, macht die interaktive Umgebung Gebrauch von Magic Commands wie `%run`. Dies korrumpierte jedoch regelmäßig den internen Typ-Zustand von *nb-mypy*. Als Workaround musste die Typprüfungs-Erweiterung manuell entladen werden, was ihren inhärenten Zweck ad absurdum führte.

Die Neuausrichtung des Projekts behält die didaktischen Vorteile der interaktiven Arbeit bei, wechselt jedoch die technologische Basis, um genau diese Brüche zu heilen. Durch den Einsatz von TypeScript in Kombination mit dem *tslab*-Kernel

wird die Typprüfung zu einem integralen Bestandteil der Ausführungsumgebung. Der TypeScript-Kernel verifiziert den Code durch seine native Kontrollflussanalyse direkt während der Eingabe und über Zellengrenzen hinweg, ohne auf fehleranfällige Zusatzskripte oder ständige Neustarts der Prüfwerkzeuge angewiesen zu sein.

3 RecursiveSet: API und fachliche Modellierung

Wie bereits in Kapitel 1 anhand von Listing 1 und Listing 2 zu sehen war, verhalten sich Mengen in Python und TypeScript unterschiedlich. Dieses Kapitel erklärt zunächst, worauf diese Unterschiede fachlich und technisch zurückzuführen sind. Anschließend wird gezeigt, wie die Bibliothek `RecursiveSet` diese Unterschiede für die Arbeit mit mathematischen Mengenstrukturen ausgleicht und dadurch eine an Python angelehnte Arbeitsweise in TypeScript ermöglicht.

3.1 Mathematische Mengen und Wohlunterscheidbarkeit

Was ist überhaupt eine Menge im mathematischen Sinn? Georg Cantor beschreibt eine Menge als „Zusammenfassung bestimmter wohlunterschiedener Objekte [...] zu einem Ganzen“ [2, S. 481]. Für die Umsetzung mengenbasierter Algorithmen in einer Programmiersprache ergibt sich daraus unmittelbar eine praktische Frage: Wann gelten zwei Elemente als gleich?

Python und TypeScript beziehungsweise JavaScript beantworten diese Frage unterschiedlich. In Python können unveränderliche, hashbare Werte wie Tupel als Elemente einer Menge verwendet werden. Zwei Tupel gelten dabei als gleich, wenn sie denselben Inhalt haben (Wertsemantik). In JavaScript werden Objekte wie Arrays in einem `Set` dagegen über ihre Referenzidentität unterschieden. Zwei Arrays mit gleichem Inhalt gelten also nur dann als gleich, wenn es sich um dieselbe physische Array-Instanz im Arbeitsspeicher handelt.

Listing 39 zeigt zunächst das Verhalten in Python:

```
1  # Python: Tupel sind unveränderlich und als Set-Elemente geeignet
2  zahlen_menge: set[tuple[int, int]] = set()
3
4  zahlen_menge.add((1, 2))
5  zahlen_menge.add((2, 1))
6
7  print(zahlen_menge)          # {(1, 2), (2, 1)}
8  print((2, 1) in zahlen_menge) # True
```

Listing 39: Mengenverhalten in Python

Die Mitgliedschaftsabfrage funktioniert hier, weil Tupel unveränderlich sind und in Mengen anhand ihrer inhaltlichen Gleichheit berücksichtigt werden.

Anders verhält es sich in TypeScript, wie Listing 40 demonstriert. Dieselbe Modellierungsidee führt zu einem anderen Ergebnis:

```

1 type ZahlenTupel = [number, number];
2 const zahlenMenge = new Set<ZahlenTupel>();
3
4 zahlenMenge.add([1, 2]);
5 zahlenMenge.add([2, 1]);
6
7 // Die Suche nach einem inhaltlich gleichen, aber neu erzeugten Array:
8 console.log(zahlenMenge.has([2, 1])); // false

```

Listing 40: Mengenverhalten in TypeScript

Hier schlägt die Abfrage fehl, weil das neu erzeugte Array `[2, 1]` in Zeile 7 eine andere Speicheradresse besitzt als das in Zeile 5 eingefügte Array.

Für Algorithmen aus der diskreten Mathematik und theoretischen Informatik ist dieser Unterschied essenziell. Da TypeScript native inhaltsbasierte Vergleiche für strukturierte Objekte nicht unterstützt, ist für deren Verarbeitung eine eigene Datenstruktur erforderlich.

3.2 Kernbausteine der Bibliothek

Um die referenzbasierte Gleichheitsprüfung der JavaScript-Laufzeitumgebung zu umgehen, stellt die Bibliothek drei Datenstrukturen bereit. Diese Bausteine garantieren die in der Mathematik vorausgesetzte Wertsemantik und bilden das Fundament, um Zustände, Übergänge und Alphabet-Zuordnungen abbilden zu können.

3.2.1 Die Datenstruktur `RecursiveSet`

Die bisherigen Beispiele haben gezeigt, dass native Mengen in TypeScript für strukturelle mathematische Modelle nur eingeschränkt geeignet sind. An dieser Stelle setzt `RecursiveSet` an. Auf API-Ebene orientiert sie sich an der aus Python vertrauten Arbeit mit Mengen: Elemente können hinzugefügt, entfernt und auf Mitgliedschaft geprüft werden. Für die Benutzung verhält sich `RecursiveSet` wie eine gewöhnliche Mengenstruktur, während sie intern die Wertsemantik sicherstellt.

Hinzufügen von Elementen

Das Einfügen neuer Elemente erfolgt in beiden Sprachen über die `add`-Methode. Listing 41 zeigt, dass sich die Benutzung deckt.

Mitgliedschaftstest

Die Prüfung auf Mitgliedschaft gehört zu den Grundoperationen. Während Python hierfür den Operator `in` verwendet, stellt `RecursiveSet` die Methode `has()` bereit

```

1 alphabet = {"a", "b", "c"}
2 alphabet.add("d")
3
4 print(alphabet) # {'a', 'b', 'c', 'd'}

```

```

1 const alphabet = new RecursiveSet<string>("a", "b", "c");
2 alphabet.add("d");
3
4 console.log(alphabet); // {a, b, c, d}

```

Listing 41: Hinzufügen von Elementen in Python und mit RecursiveSet

(siehe Listing 42).

```

1 alphabet = {"a", "b", "c"}
2
3 print("b" in alphabet) # True
4 print("x" in alphabet) # False

```

```

1 const alphabet = new RecursiveSet<string>("a", "b", "c");
2
3 console.log(alphabet.has("b")); // true
4 console.log(alphabet.has("x")); // false

```

Listing 42: Mitgliedschaftstest in Python und mit RecursiveSet

Entfernen von Elementen

Schließlich müssen Elemente auch wieder aus einer Menge entfernt werden können. Listing 43 zeigt die Anwendung beider Ansätze.

Die zentrale Stärke der Bibliothek zeigt sich jedoch erst dann, wenn nicht nur primitive Strings, sondern strukturierte und rekursive Werte als Elemente betrachtet werden.

3.2.2 Strukturierte Elemente: Die Tuple-Klasse

Neben Mengen stützen sich theoretische Modelle häufig auf Tupel. Ein Tupel ist eine endliche, geordnete Sequenz von Elementen. Im Unterschied zu Mengen ist die Reihenfolge der Einträge dabei strikt, und Mehrfachvorkommen sind zulässig. Ein Beispiel ist ein Koordinatenpaar (x, y) in einem zweidimensionalen Raum. Für zwei Punkte $P = (2, 5)$ und $Q = (5, 2)$ gilt daher mathematisch $P \neq Q$.

```

1 alphabet = {"a", "b", "c"}
2 alphabet.remove("b")
3 print(alphabet) # {'a', 'c'}

```

```

1 const alphabet = new RecursiveSet<string>("a", "b", "c");
2 alphabet.remove("b");
3 console.log(alphabet); // {a, c}

```

Listing 43: Entfernen von Elementen in Python und mit RecursiveSet

Da Python-Tupel unveränderliche Sequenzen sind, eignen sie sich sehr gut zur Modellierung solcher Datenstrukturen. Wird ein solches Modell in TypeScript mit Arrays nachgebildet, entstehen die in Abschnitt 3.1 beschriebenen Referenz-Probleme.

Die Bibliothek begegnet diesem Sachverhalt mit der `Tuple`-Klasse. Sie dient als geordneter, unveränderlicher Container für strukturierte Werte. Listing 44 zeigt dies an einem Beispiel aus der Modellierung besuchter Roboter-Positionen.

```

1 visited_coords: set[tuple[int, int]] = set()
2 visited_coords.add((2, 5))
3
4 print((2, 5) in visited_coords) # True

```

```

1 const visitedCoords = new RecursiveSet<Tuple<number, number>>>();
2 visitedCoords.add(new Tuple(2, 5));
3
4 const searchPoint = new Tuple(2, 5);
5 console.log(visitedCoords.has(searchPoint)); // true

```

Listing 44: Roboter-Koordinaten: Tuplel in Python vs. TypeScript

3.2.3 Assoziative Daten: Die `RecursiveMap`

Sobald Koordinaten oder Zustände modelliert sind, müssen diesen in vielen Algorithmen Werte zugeordnet werden. In der Informatik übernimmt diese Aufgabe eine Map (Dictionary). Mathematisch entspricht dies einer Abbildung $f: \text{Schlüssel} \rightarrow \text{Wert}$.

Werden strukturierte Werte als Schlüssel verwendet, muss der Abruf über ihren inhaltlichen Aufbau erfolgen. Die Klasse `RecursiveMap<K, V>` stellt hierfür

eine assoziative Datenstruktur bereit, deren Schlüssel dieselbe Wertsemantik verwenden wie `RecursiveSet`.

Listing 45 illustriert dies an einem einfachen Rastermodell.

```
1 grid: dict[tuple[int, int], str] = {}
2 grid[(2, 5)] = "Queen"
3
4 print(grid[(2, 5)]) # "Queen"

1 const safeGrid = new RecursiveMap<Tuple<[number, number]>, string>();
2 safeGrid.set(new Tuple(2, 5), "Queen");
3
4 const foundPiece = safeGrid.get(new Tuple(2, 5));
5 console.log(foundPiece); // "Queen"
```

Listing 45: Dictionaries in Python vs. RecursiveMap in TypeScript

3.3 Verarbeitung und Lebenszyklus strukturierter Daten

Mit den Bausteinen zur Speicherung von Werten stellt sich die Frage nach deren Manipulation. Theoretische Algorithmen erfordern es, Mengen zu kombinieren, funktional zu transformieren und ineinander zu verschachteln. Die folgenden Abschnitte beschreiben den Lebenszyklus dieser Strukturen sowie die API zur Durchführung von Mengenoperationen.

3.3.1 Der Freeze-Contract: Immutabilität sichern

Sobald Mengen selbst wieder in größere Strukturen eingebettet werden (etwa bei Potenzmengen), darf ein Element seinen inhaltlichen Zustand nachträglich nicht mehr verändern, da dies die interne Hashtabelle der Elternmenge korrumpieren würde.

In Python ist dies auf Typ-Ebene gelöst: Ein veränderliches `set` wirft einen `TypeError`, wenn man versucht, es in ein anderes Set einzufügen. Es muss manuell in ein unveränderliches `frozenset` konvertiert werden.

In TypeScript greift an dieser Stelle der sogenannte *Freeze-Contract*. Er legt fest, dass ein `RecursiveSet` ab dem Moment seiner Einbettung automatisch als logisch fixiert (frozen) gilt. Ein expliziter Typwechsel entfällt.

Listing 46 zeigt dieses Verhalten im direkten Sprachvergleich.

Soll ein fixierter Wert dennoch als Ausgangspunkt für weitere Berechnungen dienen, muss eine veränderliche Kopie erzeugt werden. Die Bibliothek stellt hierfür

```

1 inner_set = {1, 2}
2 outer_set = set()
3
4 # outer_set.add(inner_set) würde einen TypeError werfen
5 outer_set.add(frozenset(inner_set))

```

```

1 const innerSet = new RecursiveSet<number>(1, 2);
2 const outerSet = new RecursiveSet<RecursiveSet<number>>>();
3
4 outerSet.add(innerSet); // innerSet wird hier automatisch eingefroren
5
6 // innerSet.add(3); // Wirft zur Laufzeit einen Fehler (Frozen Set modified)

```

Listing 46: Der Freeze-Contract bei geschachtelten Mengen

`mutableCopy()` beziehungsweise `clone()` bereit (siehe Listing 47).

```

1 const workingCopy = innerSet.mutableCopy(); // alternativ: innerSet.clone()
2 workingCopy.add(3);
3
4 console.log(innerSet); // {1, 2} (Original bleibt fixiert)
5 console.log(workingCopy); // {1, 2, 3}

```

Listing 47: Arbeiten mit einer veränderlichen Kopie

3.3.2 Klassische Mengenoperationen

Neben der Konstruktion einzelner Mengen spielen klassische Mengenoperationen wie Vereinigung, Schnittmenge, Differenz und symmetrische Differenz eine Rolle. Während Python diese mathematischen Operationen als direkte Operatoren anbietet, wird dieselbe Logik in `RecursiveSet` über API-Aufrufe formuliert.

Vereinigung mit `union`

Die Vereinigung zweier Mengen enthält alle Elemente, die in mindestens einer der beiden Ausgangsmengen vorkommen ($A \cup B = \{x \mid x \in A \vee x \in B\}$).

Schnittmenge mit `intersection`

Die Schnittmenge enthält genau die Elemente, die in beiden Ausgangsmengen gemeinsam vorkommen ($A \cap B = \{x \mid x \in A \wedge x \in B\}$).

```

1 a = {1, 2, 3}
2 b = {3, 4, 5}
3 print(a | b) # {1, 2, 3, 4, 5}

```

```

1 const a = new RecursiveSet<number>(1, 2, 3);
2 const b = new RecursiveSet<number>(3, 4, 5);
3 console.log(a.union(b)); // {1, 2, 3, 4, 5}

```

Listing 48: Vereinigung zweier Mengen in Python und mit RecursiveSet

```

1 print(a & b) # {3}

```

```

1 console.log(a.intersection(b)); // {3}

```

Listing 49: Schnittmenge zweier Mengen in Python und mit RecursiveSet

Differenz mit difference

Die Differenz zweier Mengen enthält alle Elemente der ersten Menge, die nicht in der zweiten Menge vorkommen ($A \setminus B = \{x \mid x \in A \wedge x \notin B\}$).

```

1 print(a - b) # {1, 2}

```

```

1 console.log(a.difference(b)); // {1, 2}

```

Listing 50: Differenz zweier Mengen in Python und mit RecursiveSet

Symmetrische Differenz mit symmetricDifference

Die symmetrische Differenz enthält genau die Elemente, die in einer der beiden Mengen vorkommen, aber nicht in beiden zugleich ($A \triangle B = (A \setminus B) \cup (B \setminus A)$).

Die API in TypeScript macht durch die Methodenbenennung unmittelbar sichtbar, welche mengentheoretische Verknüpfung der Algorithmus ausführt.

3.3.3 Teil- und Obermengenbeziehungen

Eine Menge A heißt Teilmenge von B ($A \subseteq B$), wenn jedes Element aus A auch in B enthalten ist. Umgekehrt heißt A Obermenge von B ($A \supseteq B$), wenn alle Elemente von B in A enthalten sind.

```
1 print(a ^ b) # {1, 2, 4, 5}
```

```
1 console.log(a.symmetricDifference(b)); // {1, 2, 4, 5}
```

Listing 51: Symmetrische Differenz in Python und mit RecursiveSet

In Python lassen sich diese Relationen über die Operatoren `<=` und `>=` ausdrücken. Listing 52 zeigt die entsprechende Gegenüberstellung zur TypeScript-Bibliothek.

```
1 small = {2, 3}
2 large = {1, 2, 3, 4, 5}
3
4 print(small <= large) # True
5 print(large >= small) # True
6 print(large <= small) # False
```

```
1 const small = new RecursiveSet<number>(2, 3);
2 const large = new RecursiveSet<number>(1, 2, 3, 4, 5);
3
4 console.log(small.isSubset(large)); // true
5 console.log(large.isSuperset(small)); // true
6 console.log(large.isSubset(small)); // false
```

Listing 52: Teil- und Obermengenbeziehungen im Sprachvergleich

3.3.4 Auswahl eines beliebigen Elements

In mathematischen Argumentationen wird häufig ein beliebiges Element $x \in A$ betrachtet. Dies darf programmatisch nicht mit dem ersten Element einer Iteration verwechselt werden.

Da `RecursiveSet` intern eine stabile Iterationsreihenfolge besitzt, würde ein Zugriff auf das erste Element stets dasselbe Ergebnis liefern. Für die Zufallsauswahl stellt die Bibliothek daher die Methode `pickRandom()` bereit.

3.3.5 Kartesisches Produkt und Potenzmenge

Das kartesische Produkt ($A \times B$) und die Potenzmenge ($\mathcal{P}(A)$) sind Operationen, die neue Strukturen generieren. Da Python für diese Operationen keine nativen Methoden am Set-Typ anbietet, werden sie häufig über Hilfsfunktionen abgebildet.

```

1 import random
2
3 values = {1, 2, 3, 4, 5}
4 arbitrary_element = random.choice(tuple(values))

```

```

1 const values = new RecursiveSet<number>(1, 2, 3, 4, 5);
2 const arbitraryElement = values.pickRandom();

```

Listing 53: Auswahl eines Elements in Python und mit RecursiveSet

`RecursiveSet` fasst diese Schritte zusammen und stellt beide Operationen als Methoden bereit.

Kartesisches Produkt mit `cartesianProduct`

Das kartesische Produkt zweier Mengen A und B ist die Menge aller geordneten Paare ($A \times B = \{(a, b) \mid a \in A \wedge b \in B\}$).

```

1 main_dishes = {"Burger", "Pasta"}
2 drinks = {"Water", "Cola"}
3
4 meals = {(d, b) for d in main_dishes for b in drinks}

```

```

1 const mainDishes = new RecursiveSet<string>("Burger", "Pasta");
2 const drinks = new RecursiveSet<string>("Water", "Cola");
3
4 const meals = mainDishes.cartesianProduct(drinks);

```

Listing 54: Kartesisches Produkt in Python und mit RecursiveSet

Potenzmenge mit `powerset`

Die Potenzmenge einer Menge A ist die Menge aller Teilmengen von A ($\mathcal{P}(A) = \{X \mid X \subseteq A\}$). Listing 55 zeigt den `powerset()`-Aufruf.

```

1 const toppings = new RecursiveSet<string>("Cheese", "Sauce", "Olives");
2
3 const allPossibleOrders = toppings.powerset();
4 console.log(allPossibleOrders.size); // 2^3 = 8

```

Listing 55: Potenzmenge mit RecursiveSet

3.3.6 Funktionale Transformationen statt Komprehension

Für die Transformation von Mengen nutzt Python sogenannte Mengenkomprehensionen (Set Comprehensions), um Selektion und Transformation kompakt in einer Zeile zu vereinen. Listing 56 zeigt dieses Grundprinzip: Aus einer Menge von Zahlen sollen alle ungeraden Werte ausgewählt und mit 10 multipliziert werden.

```
1 mixed_numbers = {1, 2, 3, 4, 5}
2 odd_tens = {n * 10 for n in mixed_numbers if n % 2 != 0}
```

Listing 56: Python: Mengenkomprehension mit Selektion und Transformation

Da sich eine derartige Syntax in TypeScript nicht direkt als Spracherweiterung nachbilden lässt, stellt `RecursiveSet` funktionale Operationen bereit, mit denen sich dieselbe Logik durch Methodenverkettung ausdrücken lässt.

Mathematische Sicht und Arrow Functions

Zur formalen Beschreibung sei A eine Menge, $f: A \rightarrow B$ eine Funktion und $P(x)$ ein Prädikat (eine Aussage über x , die wahr oder falsch ist).

Im TypeScript-Code werden solche Logiken als „Arrow Functions“ (Lambda-Funktionen) formuliert. Die Schreibweise `n => n * 10` liest sich als „Nimm Element n und bilde es ab auf $n \cdot 10$ “. Damit lässt sich die mathematische Mengenschreibweise $\{x \in A \mid P(x)\}$ in Code überführen.

Selektion mit `filter`

Die Methode `filter` wählt aus einer Ausgangsmenge alle Elemente aus, die ein gegebenes Prädikat erfüllen ($\text{filter}(A, P) = \{x \in A \mid P(x)\}$).

```
1 const vocabulary = new RecursiveSet<string>("cat", "dog", "bat", "mouse", "owl");
2
3 const threeLetterWords = vocabulary.filter(word => word.length === 3);
```

Listing 57: Selektion mit `filter`

Abbildung mit `map`

Die Methode `map` wendet eine Funktion auf jedes Element an ($\text{map}(A, f) = \{f(x) \mid x \in A\}$). Da das Ergebnis wiederum eine Menge ist, werden doppelte Resultate zusammengefasst.

Quantoren mit `every` und `some`

Die Bibliothek unterstützt Operationen, die logischen Quantoren entsprechen. Die Methode `every` prüft, ob ein Prädikat für *alle* Elemente gilt ($\forall x \in A : P(x)$),

```

1  const uppercaseWords = threeLetterWords.map(word => word.toUpperCase());
2
3  // Verschiedene Wörter können dieselbe Länge haben.
4  // Duplikate werden vom Set zusammengefasst:
5  const wordLengths = threeLetterWords.map(word => word.length); // {3}

```

Listing 58: Abbildung mit map

während `some` die Existenz *mindestens* eines Elements testet ($\exists x \in A : P(x)$).

```

1  const numbers = new RecursiveSet<number>(2, 4, 6, 8);
2
3  const areAllEven = numbers.every(n => n % 2 === 0); // true
4  const containsEight = numbers.some(n => n === 8);   // true

```

Listing 59: Logische Quantoren mit every und some

Kombination von Selektion und Abbildung mit filterMap

Sind Selektion und Transformation gemeinsam erforderlich, bündelt `filterMap` diese in einem Durchlauf ($\text{filterMap}(A, P, f) = \{f(x) \mid x \in A \wedge P(x)\}$). Dies entspricht der Python-Komprehension aus Listing 56.

```

1  const mixedNumbers = new RecursiveSet<number>(1, 2, 3, 4, 5);
2
3  const oddTens = mixedNumbers.filterMap(
4    n => n % 2 !== 0, // Prädikat
5    n => n * 10       // Transformation
6  ); // {10, 30, 50}

```

Listing 60: Kombination aus Filter und Abbildung mit filterMap

Verschachtelte Konstruktionen mit flatMap

Wenn jedem Eingabewert eine Menge von Ergebnissen zugeordnet wird, fasst `flatMap` diese Teilmengen zu einer Gesamtmenge zusammen ($\text{flatMap}(A, f) = \bigcup_{x \in A} f(x)$). Listing 61 zeigt zunächst die verschachtelte Python-Komprehension.

In TypeScript flacht die Methode `flatMap` die entstehenden Teilmengen direkt in das aufrufende `integerSet` ab.

Aggregation mit reduce

Während die bisher betrachteten Operationen Mengen erzeugen oder logische Aussagen liefern, fasst `reduce` alle Elemente zu einem einzelnen Wert zusammen

```

1 integer_set = {0}
2 natural_numbers = [1, 2, 3]
3
4 expanded_set = {val for n in natural_numbers for val in (n, -n)}
5 integer_set |= expanded_set

```

Listing 61: Python: Verschachtelte Mengenkomprehension

```

1 const integerSet = new RecursiveSet<number>(0);
2 const naturalNumbers = [1, 2, 3];
3
4 integerSet.flatMap(
5   naturalNumbers,
6   n => new RecursiveSet<number>(n, -n)
7 );

```

Listing 62: Verschachtelte Konstruktion mit flatMap

$$(\sum_{x \in A} x).$$

```

1 const costs = new RecursiveSet<number>(10, 20, 30);
2
3 const totalSum = costs.reduce((acc, current) => acc + current, 0); // 60

```

Listing 63: Aggregation mit reduce

Die native Komprehensionssyntax wird somit durch eine API funktionaler Operationen abgebildet, deren Bedeutung sich direkt auf die mathematische Notation übertragen lässt.

4 Interne Architektur von RecursiveSet

Während Kapitel 3 die fachliche Programmierschnittstelle und die konzeptionelle Notwendigkeit einer wertsemantischen Mengenstruktur behandelte, fokussiert dieses Kapitel die zugrunde liegende technische Architektur. Dabei wird beleuchtet, wie die Bibliothek die Brücke zwischen der formalen mathematischen Theorie und der maschinennahen Ausführung in der JavaScript-Laufzeitumgebung (V8) schlägt.

4.1 Architektonische Grundentscheidungen

Die naheliegendste Frage zu Beginn einer solchen Portierung lautet: Warum wird das native `Set` von TypeScript nicht einfach in eine Wrapper-Klasse gewickelt, um das in Abschnitt 3.1 beschriebene Problem der mangelnden Wertsemantik zu umgehen? Man könnte komplexe Objekte beispielsweise in JSON-Strings umwandeln (`JSON.stringify`) und diese Strings als Repräsentation im nativen Set speichern.

Ein solcher Ansatz scheitert jedoch an zwei fundamentalen Hürden: einer semantischen und einer architektonischen.

Die **semantische Hürde** betrifft das Wesen mathematischer Mengen. Mengen sind fachlich ungeordnet: Die Menge $\{1, 2\}$ ist mathematisch absolut identisch zur Menge $\{2, 1\}$. Eine Serialisierung mittels `JSON.stringify` ist jedoch strikt sequenziell. Sie würde die Repräsentationen `"[1,2]"` und `"[2,1]"` als zwei völlig unterschiedliche Zeichenketten und damit als zwei verschiedene Elemente werten. Um dies zu umgehen, müsste jede geschachtelte Menge vor jedem Lese- oder Schreibzugriff tiefenrekursiv sortiert werden.

Dies führt unweigerlich zur **architektonischen Hürde**: Zwar ist die native `JSON.stringify`-Funktion der JavaScript-Laufzeitumgebung hochgradig optimiert, doch das erzwungene vorherige Sortieren und das ständige Erzeugen neuer String-Objekte bei jedem noch so kleinen Zugriffsobjekt erfordert enorm viel CPU-Zeit. Zudem belastet die ständige Erzeugung temporärer Strings die Speicherverwaltung massiv:

Definition 4.1. Garbage Collection (GC): Ein Prozess der Laufzeitumgebung (wie der V8-Engine), der den Arbeitsspeicher nach Objekten durchsucht, die vom Programm nicht mehr referenziert werden, um deren Speicherplatz wieder freizugeben. Werden in kurzen Abständen extrem viele temporäre Objekte (wie JSON-Strings für Suchabfragen) erzeugt, muss die GC den Programmablauf für Aufräumarbeiten pausieren („Stop-the-World“-Pausen), was zu Latenzen und Einbrüchen in der Performance führt.

Um sowohl die mathematische Gleichheit (ordnungsunabhängige Wertsemantik) als auch eine Zugriffszeit von im Durchschnitt amortisiert $\mathcal{O}(1)$ (unter der Annahme einer guten Hash-Verteilung) zu garantieren, wurde `RecursiveSet` stattdessen als vollständige Eigenimplementierung entwickelt. Nur so erhält die Bibliothek die volle Kontrolle über den Hashing-Algorithmus (der bei Mengen ordnungsunabhängig arbeiten muss) und das Speicherlayout, um unnötige Objekt-Allokationen zu vermeiden.

4.2 Die Hash-Engine (Zentrale Komponente)

Ein zentraler Bestandteil jeder Hashtabelle ist die Hash-Engine. Ihre Aufgabe ist es, aus beliebigen, komplexen Werten deterministische Zahlen (Hash-Werte) zu generieren. Dieser Hash-Wert fungiert als mathematischer Wegweiser, der den Suchraum in der Tabelle massiv eingrenzt, bevor die aufwendigere inhaltliche Gleichheitsprüfung stattfindet.

Die Bibliothek nutzt die zentrale Funktion `getHashCode` als *Dispatcher*. Ein Dispatcher (engl. für Verteiler oder Leitstelle) ist ein Architekturmuster, bei dem eine zentrale Funktion den eingehenden Datentyp untersucht und die Aufgabe an den jeweils spezialisierten, effizientesten Unteralgorithmus weiterleitet.

4.2.1 Ganzzahlen und der ArrayBuffer-Trick für IEEE-754

Wie bereits in Definition 2.1 (Kapitel 2) dargelegt, unterscheidet TypeScript auf Typ-Ebene nicht zwischen Ganzzahlen und Kommazahlen. Alle numerischen Werte werden nach dem IEEE-754-Standard als 64-Bit-Fließkommazahlen behandelt.

Für die Berechnung von Hash-Werten ist dies jedoch eine architektonische Hürde: Klassische bitweise Hash-Operationen (wie das exklusive ODER, `^`) arbeiten in JavaScript ausschließlich mit 32-Bit-Ganzzahlen. Wendet man sie direkt auf eine Fließkommazahl an, schneidet die Laufzeitumgebung die Nachkommastellen schlichtweg ab. Der Hash von `3.1` wäre demnach identisch mit dem Hash von `3.9`, was zu massiven Hash-Kollisionen führen würde.

Die Hash-Engine löst dieses Problem durch zwei strikt getrennte Ausführungspfade. Da dieser Algorithmus, siehe Listing 64, sehr dicht und hardwarenah geschrieben ist, lohnt sich eine schrittweise Betrachtung.

```
1  const _buffer = new ArrayBuffer(8);
2  const _f64 = new Float64Array(_buffer);
3  const _i32 = new Int32Array(_buffer);
4
5  // Mixing constants (derived from MurmurHash3)
6  const HASH_C1 = 0xcc9e2d51;
7  const HASH_C2 = 0x1b873593;
8
9  function getHashCode(val: Value): number {
10     if (typeof val === 'number') {
11         // Fast Path: 32-Bit Ganzzahlen
12         if ((val | 0) === val) {
13             let h = val | 0;
14             h = Math.imul(h ^ (h >> 16), 0x85ebca6b);
15             h = Math.imul(h ^ (h >> 13), 0xc2b2ae35);
16             return (h ^ (h >> 16)) >> 0;
17         }
18
19         // Slow Path: IEEE-754 Fließkommazahlen
20         _f64[0] = val;
21         const low = _i32[0];
22         const high = _i32[1];
23
24         return (Math.imul(low, HASH_C1) ^ Math.imul(high, HASH_C2)) >> 0;
25     }
26     // ...
27 }
```

Listing 64: Hashing von numerischen Werten

Schritt 1: Globale Speichervorbereitung (Zeile 1–7)

Noch bevor die eigentliche Funktion aufgerufen wird, reserviert die Bibliothek mit `new ArrayBuffer(8)` einen 8 Bytes (64 Bits) großen, rohen Block im Arbeitsspeicher. Die typisierten Arrays `_f64` und `_i32` erzeugen dabei keine neuen Daten, sondern fungieren als zwei unterschiedliche „Linsen“ (Sichten) auf exakt denselben Speicherblock. Wofür diese Linsen benötigt werden, zeigt sich im Slow Path. Zusätzlich werden die globalen Konstanten `HASH_C1` und `HASH_C2` definiert, die aus dem bewährten MurmurHash3-Algorithmus entlehnt sind [3].

Schritt 2: Der Fast Path für Ganzzahlen (Zeile 11–17)

Zunächst wird mittels der bitweisen Operation `(val | 0)` geprüft, ob die Zahl trotz des Fließkomma-Formats verlustfrei als 32-Bit-Ganzzahl behandelt werden kann.

Ist dies der Fall, wird der Wert durch sogenanntes *Avalanche-Mixing* (Lawineneffekt) vermischt. Warum ist das nötig? Würde man aufeinanderfolgende Zahlen (wie 1, 2 und 3) unverändert als Hash-Werte nutzen, entstünden in der Hashtabelle schnell sogenannte **Cluster** (Klumpen). Das bedeutet, dass viele Elemente direkt nebeneinander abgelegt werden. Beim späteren Einfügen von Elementen führte dies unweigerlich zu langen Suchen nach Plätzen, da der Algorithmus viele belegte Positionen überspringen muss, bevor er eine Lücke findet.

Um diese Klumpenbildung zu verhindern, nutzt der Code Konstanten wie `0x85ebca6b`. Das Präfix `0x` kennzeichnet dabei lediglich eine **hexadezimale** Zahl, eine kompakte Schreibweise für lange Bitmuster. Diese speziellen Bitmuster sorgen bei der Multiplikation für eine optimale Streuung: Ändert sich an der Eingabezahl nur ein einziges Bit, ändert sich durch den Lawineneffekt fast das gesamte Bitmuster des resultierenden Hash-Werts. So landen auch sehr ähnliche Zahlen auf völlig unterschiedlichen Tabellenplätzen.

Schritt 3: Hardwarenahe Multiplikation mit `Math.imul`

Auffällig ist in Zeile 14 und 15 die ständige Verwendung von `Math.imul`. Ein gewöhnliches Multiplikationszeichen (`*`) würde in JavaScript dazu führen, dass die erzeugten Bitmuster intern wieder als 64-Bit-Fließkommazahlen berechnet werden. Bei großen Multiplikatoren führt das unweigerlich zu Genauigkeitsverlusten (Rundungsfehlern) und macht die unteren Bits des Hash-Werts unbrauchbar. Die Funktion `Math.imul` (Integer Multiplication) zwingt die JavaScript-Engine stattdessen zu einer echten, hardwarenahen 32-Bit-Ganzzahl-Multiplikation. Bits, die durch einen Überlauf über die 32-Bit-Grenze hinausschießen, werden auf CPU-Ebene einfach abgeschnitten, was für Hashing-Algorithmen das gewünschte Verhalten ist.

Schritt 4: Der Slow Path für Fließkommazahlen (Zeile 19–24)

Schlägt die Ganzzahl-Prüfung fehl, greift der Slow Path. Hier kommt nun ein architektonischer Ansatz zum Einsatz, der an ein `union`-Konstrukt aus der Sprache C erinnert. Um die gesamten 64 Bits der Fließkommazahl (Vorzeichen, Exponent und Mantisse) für das Hashing zu nutzen, müssen diese in zwei 32-Bit-Hälften zerlegt werden.

Anstatt die Zahl durch teure mathematische Operationen zu zerspalten, greift die Bibliothek auf den in Schritt 1 vorbereiteten `ArrayBuffer` zurück:

1. Die Fließkommazahl wird durch die 64-Bit-Linse (`_f64[0] = val`) in Zeile 20

nativ in den Speicher geschrieben.

2. Anschließend werden in den Zeilen 21 und 22 exakt dieselben 64 Bits durch die Ganzzahl-Linse (`_i32`) ohne jegliche Konvertierung als zwei fertige 32-Bit-Blöcke (`low` und `high`) wieder ausgelesen.

Dieses direkte Lesen und Schreiben von Speicherbits erzeugt keine temporären Objekte und entlastet den Garbage Collector somit vollständig. Zum Abschluss werden beide Hälften in Zeile 24 über `Math.imul` mit den globalen Konstanten `HASH_C1` und `HASH_C2` überkreuz multipliziert und per exklusivem ODER vereint. Dadurch ist sichergestellt, dass selbst winzigste Änderungen im Exponenten oder in der Mantisse der Fließkommazahl einen neuen Hash-Wert generieren.

4.2.2 Hashing von Strings und strukturellen Objekten

Bei Zeichenketten und komplexen Objekten greifen andere Mechanismen, die ebenfalls auf Performance und Speicherschonung ausgelegt sind, wie in Listing 65 dargestellt.

```
1   if (typeof val === 'string') {
2       let h = 0x811c9dc5;
3       const len = val.length;
4       for (let i = 0; i < len; i++) {
5           h ^= val.charCodeAt(i);
6           h = Math.imul(h, 0x01000193);
7       }
8       return h >>> 0;
9   }
10
11   if (val && typeof val === 'object') {
12       return val.hashCode();
13   }
```

Listing 65: Hashing von Strings und strukturellen Objekten

Für **Strings** verwendet die Bibliothek eine Variante des *FNV-1a*-Algorithmus (Fowler–Noll–Vo). Die Zeichenkette wird in einer Schleife durchlaufen und der numerische Wert jedes Zeichens (`charCodeAt`) wird über ein exklusives ODER (`^=`) in den Hash eingemischt. Der bewusste Verzicht auf Methoden, die den String in Arrays aufspalten oder neue Teilzeichenketten (Sub-Strings) erzeugen, stellt sicher, dass während der Schleife keine temporären Objekte im Arbeitsspeicher reserviert (alloziert) werden müssen. Diese sogenannte *Allokationsfreiheit* entlastet den Garbage Collector.

Bei **komplexen Objekten** (wie verschachtelten Mengen oder Tupeln) delegiert die Hash-Engine die Verantwortung an das Objekt selbst, indem sie schlicht `val.hashCode` abrufen.

Damit der TypeScript-Compiler garantieren kann, dass diese Eigenschaft auf dem übergebenen Objekt überhaupt existiert, stützt sich die Bibliothek auf einen vertraglichen Rahmen: das interne `Structural`-Interface. Dieses ist in Listing 66 dargestellt.

```
1 interface Structural {
2     /**
3      * Ein stabiler Hash-Wert für das Objekt.
4      * Der Zugriff auf diese Eigenschaft friert den Zustand des Objekts
5      * in rekursiven Strukturen in der Regel ein.
6      */
7     readonly hashCode: number;
8
9     /**
10     * Prüft die inhaltliche (strukturelle) Gleichheit mit einem anderen Objekt.
11     */
12     equals(other: unknown): boolean;
13
14     /** * Liefert eine deterministische Textrepräsentation.
15     */
16     toString(): string;
17 }
```

Listing 66: Der Vertrag für komplexe Mengenelemente

Wie in Abschnitt 2.2.3 im Rahmen des statischen Duck-Typings erläutert, fordert TypeScript keine starren Klassenhierarchien. Jedes Objekt, das die in Listing 66 definierten Eigenschaften und Methoden bereitstellt, wird vom Typsystem automatisch als `Structural` anerkannt und kann in der Bibliothek als Schlüssel oder Mengenelement verwendet werden.

Dieser Vertrag definiert exakt die drei Säulen, die eine wertebasierte Hashtabelle benötigt:

1. `hashCode`: Liefert den vorberechneten Hash-Wert zur schnellen Eingrenzung des Suchraums. Das Schlüsselwort `readonly` garantiert dem Typsystem, dass sich dieser Wert nach der Erstellung nicht mehr ändert. Der lesende Zugriff auf diese Eigenschaft ist zudem der Auslöser für den kaskadierenden Freeze-Mechanismus (siehe Abschnitt 4.7).
2. `equals`: Übernimmt die inhaltliche Gleichheitsprüfung, sobald der Hash-Wert eine mögliche Kollision oder einen Treffer anzeigt (siehe Abschnitt 4.4).

3. `toString`: Erzwingt eine konsistente Textrepräsentation, was für die deterministische Konsolenausgabe von tief verschachtelten Strukturen essenziell ist.

Genau dieses Interface ist der Grund, warum sich die in Kapitel 3 vorgestellten Klassen `Tuple` und `RecursiveMap` nahtlos, typsicher und ohne zusätzlichen Konvertierungsaufwand in die Hash-Architektur einfügen lassen.

4.3 Speicherlayout (Structure of Arrays)

Ein klassischer, objektorientierter Entwurf für eine Hashtabelle würde Objekte der Form `{ value, hash }` anlegen. Diesen Ansatz nennt man *Array of Structures* (AoS). Bei Tausenden von Zuständen in einem Automaten führt dies jedoch dazu, dass der Arbeitsspeicher fragmentiert. Die CPU muss ständig an verschiedene Orte im RAM springen, um die Objekte zu lesen, was zusätzliche Taktzyklen erfordert.

Um die Cache-Lokalität der CPU besser auszunutzen, wird von `RecursiveSet` ein Entwurfsmuster aus dem High-Performance-Computing implementiert:

Definition 4.2. Structure of Arrays (SoA): Bei diesem Architekturmuster werden Objekte nicht als zusammenhängende Datenblöcke im Speicher abgelegt. Stattdessen werden ihre einzelnen Eigenschaften in voneinander getrennten, parallelen Arrays gespeichert. Dies verbessert die Cache-Lokalität, da die CPU beim Durchsuchen einer bestimmten Eigenschaft (z. B. aller Hash-Werte) diese als kompakten Block am Stück aus dem Arbeitsspeicher (RAM) in den schnellen CPU-Cache laden kann [4, S. 161].

```
1 class RecursiveSet<T extends Value> implements Structural, Iterable<T> {
2     // Dichte Arrays (Nutzdaten liegen ohne Lücken nebeneinander)
3     private _values: T[] = [];
4     private _hashes: number[] = [];
5
6     // Indextabelle für die Suchstruktur (Open Addressing)
7     private _indices: Uint32Array;
8
9     private _bucketCount: number;
10    private _mask: number;
11 }
```

Listing 67: Speicherarchitektur im Structure of Arrays (SoA) Format

Um die Effizienz dieser Architektur zu verstehen, müssen die fünf Felder der Klasse getrennt betrachtet werden. Sie teilen sich in zwei logische Ebenen auf: den eigentlichen Datenbestand (die dichten Arrays) und die Navigationsstruktur (die Indextabelle).

Ebene 1: Der dichte Datenbestand

Die Arrays `_values` und `_hashes` wachsen dicht (ohne Lücken) heran. Wenn die Menge drei Elemente enthält, sind exakt die Indizes 0, 1 und 2 belegt.

- `_values`: Speichert die tatsächlichen, typisierten Elemente der Menge.
- `_hashes`: Speichert die zugehörigen, vorberechneten 32-Bit-Hash-Werte. Das Element an Index 5 in `_values` gehört exakt zum Hash an Index 5 in `_hashes`. Dass die Hashes in einem eigenen Array liegen, beschleunigt die Suche, da die CPU Hash-Werte auf Kollisionen scannen kann, ohne das speicherintensive `_values`-Array laden zu müssen.

Ebene 2: Die Navigationsstruktur

Während die dichten Arrays lediglich die Daten bunkern, kümmert sich die zweite Ebene darum, diese Daten über Hash-Werte effizient wiederzufinden:

- `_indices`: Dies ist die eigentliche Hashtabelle (die Suchstruktur). Sie ist als `UInt32Array` implementiert und funktioniert wie ein Inhaltsverzeichnis. Bemerkenswert ist hierbei, dass sie **keine echten Elemente** speichert, sondern lediglich **Verweise auf Positionen** in den dichten Arrays.

Da ein neu angelegtes `UInt32Array` im Arbeitsspeicher standardmäßig mit Nullen gefüllt ist, nutzt die Bibliothek den Wert 0 als zweckmäßige Markierung für einen „leeren Slot“. Um nun den ersten echten Index (die 0) der dichten Arrays davon unterscheiden zu können, werden alle gespeicherten Positionen intern um den Wert 1 erhöht. Ein Eintrag von 3 in der Indextabelle bedeutet also: „Ein *möglicher* Treffer für diesen Hash liegt in den dichten Arrays an Position 2“. Ob es sich dabei wirklich um das gesuchte Element oder nur um eine Kollision handelt, und wie der Algorithmus weiter navigiert, wird im Abschnitt 4.5 (*Linear Probing*) detailliert beleuchtet.

- `_bucketCount`: Definiert die Gesamtkapazität der `_indices`-Tabelle (die Anzahl der verfügbaren Such-Slots). Diese Kapazität ist zwingend immer eine Zweierpotenz (z. B. 16, 32, 64).
- `_mask`: Die Bitmaske ist ein klassischer Performance-Trick aus der Hardware-Programmierung. Um einen beliebig großen Hash-Wert auf die tatsächliche Größe der Tabelle herunterzuberechnen, nutzt man klassischerweise

die Modulo-Division (`hash % _bucketCount`). Da Divisionen für die CPU jedoch vergleichsweise teuer sind, wird die Bitmaske auf `_bucketCount - 1` gesetzt (bei 16 Buckets ist die Maske 15, binär 00001111). Die Operation `hash & _mask` (bitweises UND) liefert exakt dasselbe Ergebnis wie Modulo, wird von der CPU als elementare Bit-Operation aber typischerweise in einem einzigen Taktzyklus berechnet.

4.4 Konzepte der Gleichheit und Type Narrowing

Wie in Abschnitt 4.2 beschrieben, dient der Hash-Wert lediglich als Wegweiser, um die Suche in der Hashtabelle einzugrenzen. Da zwei inhaltlich unterschiedliche Mengen rein rechnerisch denselben Hash-Wert generieren können (Kollision), muss die Datenstruktur gefundene Kandidaten iterativ auf Gleichheit prüfen. Hierbei ist eine präzise terminologische Trennung essenziell:

- **Referenzidentität:** Zwei Variablen verweisen auf exakt denselben Bereich im Arbeitsspeicher (`a === b`).
- **Hash-Gleichheit:** Zwei Objekte generieren denselben numerischen Hash-Wert (`a.hashCode === b.hashCode`). Dies ist eine notwendige, aber keine hinreichende Bedingung für weitere Gleichheitsannahmen.
- **Strukturelle Gleichheit:** Zwei getrennte Objekte weisen denselben inhaltlichen Aufbau auf (geprüft über die Methode `equals`).
- **Mathematische Gleichheit:** Die semantische Eigenschaft, dass zwei Mengen dieselben Elemente beinhalten, unabhängig von der Reihenfolge, in der sie im Speicher abgelegt wurden.

Genau diese Aufgabe übernimmt die vertraglich im `Structural` -Interface geforderte `equals` -Methode. Am Beispiel der Klasse `RecursiveSet` zeigt sich, wie TypeScript-spezifische Typsicherheit und Performance-Optimierungen hier Hand in Hand gehen:

Dieser Algorithmus ist wie ein Trichter aufgebaut: Er versucht, eine aufwendige Tiefenprüfung (Zeile 13–15) durch immer strengere, aber effiziente Vorab-Ausschlusskriterien zu vermeiden (*Short-Circuit Evaluation*).

Sicherheit durch `unknown` und Type Narrowing (Zeile 1–6)

Die Signatur der Methode fordert bewusst den Typ `unknown` für den Parameter `other` . Wie in Definition 2.10 (Kapitel 2) erläutert, zwingt dieser Typ den Compiler dazu, jegliche blinde Interaktion mit dem Objekt zu verbieten. Die Methode weiß

```

1      equals(other: unknown): boolean {
2          // 1. Identitäts-Kurzschluss (Referenzidentität)
3          if (this === other) return true;
4
5          // 2. Typprüfung und Type Narrowing
6          if (!(other instanceof RecursiveSet)) return false;
7
8          // 3. O(1) Performance-Schranken (Größe und Hash-Gleichheit)
9          if (this.size !== other.size) return false;
10         if (this.hashCode !== other.hashCode) return false;
11
12         // 4. Tiefe strukturelle und mathematische Prüfung
13         for (const v of this._values) {
14             if (!other.has(v)) return false;
15         }
16
17         return true;
18     }

```

Listing 68: Die equals-Methode von RecursiveSet

an diesem Punkt nicht, ob `other` eine Zahl, ein String oder ein komplexes Objekt ist.

Zunächst greift ein schneller Abgleich der **Referenzidentität** (Zeile 3). Zeigen `this` und `other` auf exakt dieselbe Speicheradresse (`===`), ist jede weitere Prüfung obsolet – ein Objekt ist immer gleich zu sich selbst.

Ist dies nicht der Fall, folgt in Zeile 6 der entscheidende Schritt für die Typsicherheit: Die Laufzeitprüfung `other instanceof RecursiveSet`. Ist das Fremdobjekt gar keine Menge, wird sofort `false` zurückgegeben.

Wird dieser Check jedoch passiert, greift das in Definition 2.14 (Kapitel 2) beschriebene **Type Narrowing**. Der TypeScript-Compiler analysiert den Kontrollfluss und weiß ab Zeile 9 mit Sicherheit, dass `other` eine Instanz von `RecursiveSet` ist. Ab hier schaltet der Compiler den typsicheren Zugriff auf Eigenschaften wie `other.size`, `other.hashCode` und die Methode `other.has()` frei.

$\mathcal{O}(1)$ Performance-Schranken (Zeile 8–10)

Bevor die Methode die Iteration über alle Elemente startet, werden zwei Eigenschaften verglichen, die in konstanter Zeit ($\mathcal{O}(1)$) abrufbar sind:

1. **Größe (size):** Mengen mit unterschiedlich vielen Elementen können per Definition nicht gleich sein.
2. **Hash-Gleichheit (hashCode):** Wenn zwei `RecursiveSet`-Instanzen inhaltlich gleich sind, *müssen* sie zwingend denselben Hash-Wert besitzen. Unter-

scheiden sich die Hashes, kann die Prüfung sofort abgebrochen werden. Da der Hash-Wert der Menge intern gecacht wird (siehe Abschnitt 4.7), erfordert dieser Vergleich nahezu keine Rechenzeit.

Strukturelle und mathematische Prüfung über dichte Arrays (Zeile 12–16)

Erst wenn das Fremdobjekt *sicher* eine Menge gleicher Größe und mit demselben Hash-Wert ist, greift die Methode auf das in Abschnitt 4.3 etablierte SoA-Layout zurück. Die `for`-Schleife iteriert über das dichte `_values`-Array und prüft mittels `other.has(v)`, ob jedes Element der aktuellen Menge auch in der anderen Menge existiert.

Da die Methode `has()` selbst im Durchschnitt amortisiert in $\mathcal{O}(1)$ arbeitet (unter Annahme einer gleichmäßigen Hash-Verteilung), bleibt auch die gesamte **strukturelle und mathematische Gleichheitsprüfung** zweier Mengen im Normalfall sehr effizient und steigt nur im Worst-Case auf einen Aufwand von $\mathcal{O}(N)$.

4.5 Kollisionsauflösung und Index-Verwaltung

Da der Hash-Wert zur Indexberechnung genutzt wird, lässt es sich mathematisch nicht vermeiden, dass zwei unterschiedliche Werte denselben Index in der `_indices`-Tabelle berechnen (das sogenannte Taubenschlagprinzip). Die Datenstruktur muss diese Hash-Kollisionen auflösen.

4.5.1 Open Addressing mit Linear Probing

Um die Cache-Lokalität des SoA-Layouts beizubehalten, wird auf externe Zeigerstrukturen verzichtet und stattdessen eine flache Tabellenverwaltung genutzt:

Definition 4.3. Open Addressing mit Linear Probing (Lineares Sondieren):

Bei diesem Verfahren werden alle Verweise direkt im Array der Hashtabelle gespeichert. Berechnet der Hash einen Index (Slot), der bereits von einem anderen Element belegt ist (eine Kollision), rückt der Algorithmus schrittweise zum nächstmöglichen benachbarten Slot weiter, bis er eine Lücke findet oder den gesuchten Wert entdeckt [5, 293f].

Man kann sich dies wie das Einsortieren von Büchern in einem langen Regal vorstellen: Berechnet das System für ein neues Buch einen Stammplatz, der bereits belegt ist, wird das Buch nicht in einem externen Anbau ausgelagert. Stattdessen prüft man einfach den direkten Nachbarplatz auf der rechten Seite, bis man auf eine Lücke trifft.

Die Methode `has` aus Listing 69 demonstriert diese Suchlogik im Zusammenspiel mit dem zuvor eingeführten SoA-Speicherlayout.

```

1  has(element: T): boolean {
2      const h = getHashCode(element);
3      let idx = h & this._mask; // Start-Slot berechnen
4
5      while (true) {
6          const entry = this._indices[idx];
7          if (entry === 0) return false; // Lücke -> Element fehlt
8
9          const valIndex = entry - 1; // 1-basierten Verweis umrechnen
10
11         // 1. Hash vergleichen (schnell), 2. equals vergleichen (teuer)
12         if (this._hashes[valIndex] === h &&
13             areEqual(this._values[valIndex], element)) {
14             return true; // Treffer
15         }
16
17         // Kollision -> Gehe zum nächsten Slot (mit Wrap-around)
18         idx = (idx + 1) & this._mask;
19     }
20 }

```

Listing 69: Linear Probing am Beispiel der `has`-Methode

Dieser Suchalgorithmus ist der Programmabschnitt, der zur Laufzeit am häufigsten ausgeführt wird. Da diese Suchlogik die Basis für nahezu jede Interaktion mit der Hashtabelle bildet, sei es beim Hinzufügen (`add`), Entfernen (`remove`) oder beim reinen Nachschlagen (`has`), ist sie auf Effizienz ausgelegt. Sie kommt (mit Ausnahme des Aufrufs von `areEqual` bei einem Kandidaten) ohne weitere Funktionsaufrufe aus. Der Ablauf lässt sich in vier Schritte zerlegen:

Schritt 1: Den Start-Slot berechnen (Zeile 2–3)

Zunächst wird der 32-Bit-Hash des gesuchten Elements berechnet. Da dieser Wert weit außerhalb der Tabellengröße liegen kann, wird er mittels der Bitmaske (`h & this._mask`) auf einen gültigen Start-Index (`idx`) in der `_indices`-Tabelle heruntergebrochen.

Schritt 2: Die Abbruchbedingung der Schleife (Zeile 5–7)

Die `while (true)`-Schleife wirkt potenziell endlos. Hier greift jedoch eine Eigenschaft der Hashtabelle: Die Bibliothek stellt sicher, dass die Tabelle niemals zu mehr als 75 % gefüllt ist (der sogenannte Load-Factor, siehe Abschnitt 4.6). Es existiert also *immer* mindestens ein leerer Slot.

Wird ein Slot mit dem Wert 0 (leer) gefunden, stoppt die Suche sofort und liefert `false`. Warum? Weil das Einsortieren im Regal (*Linear Probing*) garantiert,

dass bei einer Kollision alle betroffenen Elemente nahtlos hintereinander abgelegt werden. Eine Lücke bedeutet zwingend, dass der Suchpfad an dieser Stelle endet und das gesuchte Element folglich nicht in der Menge existiert.

Schritt 3: Dereferenzierung und Hash-Vorabgleich (Zeile 9–15)

Ist der Slot belegt (`entry > 0`), wird der 1-basierte Verweis durch `entry - 1` in den korrekten 0-basierten Index (`valIndex`) für die dichten Nutzdaten-Arrays umgerechnet.

Nun zeigt sich ein Vorteil des SoA-Layouts: Anstatt sofort die `areEqual` -Funktion aufzurufen, wird zunächst nur der gespeicherte Hash-Wert (`this._hashes[valIndex]`) mit dem gesuchten Hash (`h`) verglichen. Stimmen die Hashes nicht überein, liegt eine klassische Kollision vor und die Inhaltsprüfung wird übersprungen. Nur wenn die Hashes identisch sind (**Hash-Gleichheit**), wird `areEqual` zur Verifizierung der **strukturellen Gleichheit** aufgerufen.

Schritt 4: Linear Probing und der Wrap-around (Zeile 18)

Stellte sich der belegte Slot als Kollision heraus, muss der Algorithmus den nächsten benachbarten Slot prüfen. Die Anweisung `idx = (idx + 1) & this._mask` inkrementiert den Index nicht nur um 1, sondern sorgt gleichzeitig für den sogenannten **Wrap-around**. Erreicht der Suchlauf das physische Ende des Arrays (das Ende des Regals), setzt das bitweise UND (`& this._mask`) den Wert vollautomatisch auf 0 zurück. Der Algorithmus springt also an den Anfang der Tabelle und setzt die Suche dort fort, ohne dass `if` -Abfragen für Array-Grenzen benötigt werden.

4.5.2 Löschvorgang: Tombstones und Backshift Deletion

Das Entfernen von Elementen aus einer Hashtabelle mit *Open Addressing* ist komplexer als bei verketteten Listen. Nimmt man ein Element einfach aus seinem Slot im Regal (durch Zurücksetzen auf 0), entsteht eine harte Lücke. Dies hat Folgen für das *Linear Probing*: Suchen wir nach einem Element, das aufgrund früherer Kollisionen hinter dieser neuen Lücke einsortiert wurde, bricht die `while` -Schleife an der leeren Stelle ab und meldet, das Element existiere nicht. Die Sondierungskette ist gerissen.

Der klassische Lösungsansatz für dieses Problem ist die Verwendung sogenannter Tombstones:

Definition 4.4. Tombstones (Grabsteine): Anstatt einen Slot im Regal wirklich zu leeren (0), wird ein spezielles Marker-Objekt (der Tombstone) an dieser Stelle platziert. Dieser Marker signalisiert dem Suchalgorithmus: „Hier wurde zwar etwas gelöscht, aber brich die Suche nicht ab, sondern suche im nächsten Slot weiter.“

Obwohl Tombstones einfach zu implementieren sind, führen sie bei vielen Einfüge- und Löschvorgängen dazu, dass die Tabelle über die Zeit mit Grabsteinen „verschmutzt“. Die Suchwege werden künstlich in die Länge gezogen, was die Zugriffszeiten verschlechtert. Zudem müssen Tombstones regelmäßig durch vollständige Neu-Aufbauten der Tabelle (Rehashing) entfernt werden.

Die Lösung: Backshift Deletion

Um dies zu vermeiden, nutzt die Bibliothek die *Backshift Deletion* (Rückwärts-Verschiebung) – eine Technik, die auf dem *Robin Hood Hashing* aufbaut [6].

Wird ein Element gelöscht, entsteht zunächst eine Lücke. Der Algorithmus, siehe Listing 70, blickt nun in die nachfolgenden Slots und prüft, ob Elemente weiter hinten im Regal stehen, die eigentlich weiter vorne (also näher an der Lücke oder genau darin) stehen müssten. Ist das der Fall, rücken diese Elemente auf, und die Lücke wandert weiter nach hinten, bis sie das Ende der Kollisionskette erreicht und geleert werden kann.

```

1 private removeIndex(holeIdx: number): void {
2     let i = (holeIdx + 1) & this._mask;
3
4     while (this._indices[i] !== 0) {
5         const entry = this._indices[i];
6         const valPtr = entry - 1;
7         const h = this._hashes[valPtr];
8
9         const idealIdx = h & this._mask;
10
11         const distToHole = (holeIdx - idealIdx + this._bucketCount) & this._mask;
12         const distToI = (i - idealIdx + this._bucketCount) & this._mask;
13
14         if (distToHole < distToI) {
15             this._indices[holeIdx] = entry;
16             holeIdx = i;
17         }
18         i = (i + 1) & this._mask;
19     }
20     this._indices[holeIdx] = 0;
21 }

```

Listing 70: Reparatur der Sondierungskette (Backshifting)

Dieser Algorithmus ist ein typisches Beispiel für hardwarenahe Optimierung in TypeScript. Da er ohne Funktionsaufrufe auskommt, übernimmt die Bit-Mathematik die Steuerung. Der Ablauf lässt sich in fünf Phasen unterteilen:

1. **Start der Sondierung (Zeile 2):** Der Algorithmus startet nicht bei der Lücke (`holeIdx`), sondern prüft direkt den ersten rechten Nachbarn. Das bitweise UND (`& this._mask`) sichert auch hier den zirkulären Wrap-around ab, falls die Lücke am äußersten Ende des Arrays lag.
2. **Abbruchbedingung und Dereferenzierung (Zeile 4–7):** Die Schleife iteriert, bis sie auf einen nativ leeren Slot (0) trifft. Ist ein Slot belegt, wird der 1-basierte Verweis (`entry`) wieder in den 0-basierten Index (`valPtr`) für das dichte `_hashes`-Array umgerechnet, um den Hash-Wert des aktuellen Elements auszulesen.
3. **Ideal-Slot und zirkuläre Distanz-Metrik (Zeile 9–13):** In Zeile 9 wird berechnet, an welchem Index das Element ursprünglich hätte landen sollen (`idealIdx`). Danach folgt der mathematische Kern: Wie weit ist das Element aktuell von seinem Ideal-Slot entfernt (`distToI`) und wie weit wäre es entfernt, wenn es in die Lücke rutschen würde (`distToHole`)?

Die Addition von `this._bucketCount` ist hier essenziell: Da das Array zirkulär ist, könnte `holeIdx - idealIdx` negativ werden. In JavaScript würde der Modulo-Operator bei negativen Zahlen ungeeignete Ergebnisse liefern. Die Addition stellt sicher, dass der Wert positiv bleibt, bevor die Bitmaske ihn wieder in den zirkulären Ring umrechnet.

4. **Die Robin-Hood-Bedingung (Zeile 15–18):** Nun greift die eigentliche *Backshift*-Logik. Ist die Distanz zur Lücke kleiner als die aktuelle Distanz, also gilt `distToHole < distToI`, profitiert das Element von einem Umzug. Es wird physisch in die Lücke verschoben (Zeile 16). Die Lücke wandert dadurch auf die alte Position des Elements (`holeIdx = i`) und wird im nächsten Schleifendurchlauf weiter nach hinten gereicht.
5. **Abschluss (Zeile 21):** Sobald die Schleife auf einen nativ leeren Slot trifft, ist die Kette repariert. Die nach hinten durchgereichte Lücke befindet sich nun am Ende der Sondierungskette und kann mit 0 (leer) überschrieben werden.

Nutzdaten entfernen: Der Dense-Swap

Nachdem die `removeIndex`-Methode die Sondierungskette repariert hat, muss das physische Element aus den dichten Nutzdaten-Arrays (`_values` und `_hashes`) gelöscht werden, ohne Lücken zu hinterlassen.

Ein nativer Aufruf wie `this._values.splice()` würde das Element zwar entfernen, müsste danach aber alle nachfolgenden Elemente im Arbeitsspeicher um einen Platz nach vorne rücken. Dies würde zu einer linearen Laufzeit von $\mathcal{O}(N)$ führen.

Stattdessen nutzt die Bibliothek den **Dense-Swap** (Tausch im dichten Array). Das zu löschende Element wird mit dem *allerletzten* Element des Arrays überschrieben. Danach wird das Array mittels `pop()` in konstanter Zeit ($\mathcal{O}(1)$) am Ende um einen Platz verkürzt. Da sich durch diesen Tausch die physische Position des letzten Elements geändert hat, muss abschließend nur noch der zugehörige Verweis in der Indextabelle auf den neuen Index aktualisiert werden.

Durch diese Kombination aus *Backshift Deletion* und *Dense-Swap* garantiert die Bibliothek eine stabile Performance bei Löschvorgängen.

4.6 Dynamische Kapazitätsverwaltung (Resizing und Rehashing)

Damit das *Linear Probing* (Abschnitt 4.5.1) performant bleibt und die `while`-Schleife nicht endlos läuft, muss ein gewisser Anteil der Indextabelle leer bleiben.

Je voller die Tabelle wird, desto wahrscheinlicher werden Kollisionen und desto länger werden die Sondierungsketten.

Gesteuert wird dies über den sogenannten *Load Factor*:

Definition 4.5. Load Factor (Füllgrad): Der Load Factor beschreibt das Verhältnis von gespeicherten Elementen zur Gesamtkapazität der Hashtabelle ($N/\text{Capacity}$). Die Bibliothek definiert eine feste Obergrenze von 0.75 (75 %). Sobald ein neues Element diesen Schwellenwert überschreitet, verdoppelt die Tabelle ihre Kapazität.

Gleichzeitig arbeitet die Bibliothek speicherschonend: Fällt der Füllgrad durch Löschvorgänge unter 0.25 (25 %), wird die Tabellengröße halbiert (*Shrinking*), sofern eine Mindestgröße (z. B. 16 Slots) nicht unterschritten wird. Da die Kapazität immer verdoppelt oder halbiert wird, bleibt sie stets eine strikte Zweierpotenz, was die Voraussetzung für den Einsatz der Bitmaske (`_mask`) ist.

Das Rehashing: Ein Umzug ohne Inhaltskontrolle

Wenn die Tabelle wächst oder schrumpft, können die alten Indizes nicht direkt kopiert werden. Da sich die Kapazität (`_bucketCount`) ändert, ändert sich unweigerlich auch die Bitmaske (`_mask`). Ein Hash-Wert, der zuvor auf Slot 3 abgebildet wurde, könnte in der neuen Tabelle auf Slot 19 verweisen.

Die Suchstruktur muss daher neu berechnet werden (Rehashing):

Dieser Algorithmus verdeutlicht einen architektonischen Vorteil des *Structure of Arrays*-Layouts (SoA). Da die Nutzdaten (`_values` und `_hashes`) als dicht gepackte Arrays vorliegen, müssen sie beim Rehashing nicht verschoben werden. Der Umzug betrifft ausschließlich die flache `_indices`-Tabelle.

Zudem profitiert der Code von einem signifikanten Performance-Vorteil: Da das `_values`-Array per Definition bereits bereinigt ist und ausschließlich eindeutige Elemente enthält, kann der Algorithmus die `areEqual`-Aufrufkette vollständig ignorieren. Er prüft bei einer Kollision lediglich, ob der Slot leer ist (`!= 0`), und rückt weiter.

Amortisierte Laufzeit

Auf den ersten Blick verletzt das Rehashing das Versprechen einer Hashtabelle, Elemente im Durchschnitt in $\mathcal{O}(1)$ einzufügen. Muss die Tabelle wachsen, erfordert die `for`-Schleife einen Aufwand von $\mathcal{O}(N)$, da alle N Elemente neu einsortiert werden.

In der Informatik wird dieses Verhalten über die *amortisierte Laufzeitanalyse* bewertet:

```

1     private resize(newCapacity: number): void {
2         this._bucketCount = newCapacity;
3         this._mask = newCapacity - 1;
4
5         // Frische Indextabelle anlegen (standardmäßig mit Nullen gefüllt)
6         this._indices = new Uint32Array(newCapacity);
7
8         // Alle existierenden Elemente neu einsortieren
9         for (let i = 0; i < this._values.length; i++) {
10             const h = this._hashes[i];
11             let idx = h & this._mask; // Neuen Ideal-Slot berechnen
12
13             // Vereinfachtes Linear Probing ohne equals-Prüfung
14             while (this._indices[idx] !== 0) {
15                 idx = (idx + 1) & this._mask;
16             }
17
18             this._indices[idx] = i + 1; // 1-basierten Verweis speichern
19         }
20     }

```

Listing 71: Der Rehashing-Prozess bei einer Kapazitätsänderung

Definition 4.6. Amortisierte Laufzeit: Dieser Begriff beschreibt den durchschnittlichen Aufwand einer Operation über eine sehr lange Sequenz von Aufrufen. Da die Tabelle ihre Größe jedes Mal verdoppelt, passieren Resizing-Operationen seltener. Die vereinzelt $\mathcal{O}(N)$ -Aufwände „amortisieren“ (verteilen) sich auf die weitaus zahlreicheren, sehr effizienten $\mathcal{O}(1)$ -Einfügungen. Im Durchschnitt bleibt der Aufwand für eine einzelne Einfügung somit bei amortisiert $\mathcal{O}(1)$.

Die Kombination aus Verdoppeln beim Wachsen, Halbieren beim Schrumpfen und Verzicht auf Inhaltsprüfungen während des Rehashings garantiert architektonisch ein Gleichgewicht zwischen Speicherhaushalt und Zugriffsgeschwindigkeit.

4.7 Technische Umsetzung des Freeze-Contracts

In Abschnitt 3.3.1 wurde der *Freeze-Contract* aus API-Sicht eingeführt. Dieser besagt, dass Mengen aus Sicherheitsgründen unveränderlich werden, sobald sie in eine andere Struktur geschachtelt werden. Technisch wird dieser Mechanismus über die Abfrage des Hash-Werts gesteuert.

Da `RecursiveSet` eine ungeordnete Menge repräsentiert, darf der Hash-Wert nicht von der historischen Einfügereihenfolge abhängen. Dies wird durch eine

fortlaufende XOR-Verknüpfung (`_xorHash ^= h;`) beim Einfügen sichergestellt, da die XOR-Operation kommutativ ist.

Der tatsächliche Freeze-Vorgang wird ausgelöst, sobald eine übergeordnete Struktur den Getter `get hashCode()` der Kind-Menge aufruft:

```
1  get hashCode(): number {
2      if (this._isFrozen) return this._xorHash;
3
4      // Iteriert über alle Elemente und berührt deren hashCode,
5      // wodurch der Freeze-Effekt nach unten durchgereicht wird.
6      for (let i = 0; i < this._values.length; i++) {
7          const v = this._values[i];
8          if (v && typeof v === 'object') {
9              v.hashCode;
10         }
11     }
12
13     this._isFrozen = true;
14     return this._xorHash;
15 }
```

Listing 72: Einfrieren durch Hash-Abruf

Sobald `this._isFrozen` auf `true` gesetzt ist, blockieren die Methoden `add` und `remove` mit einer Fehlermeldung.

Die `for`-Schleife ruft proaktiv `v.hashCode` auf allen gespeicherten Objekten auf. Dies erzeugt einen kaskadierenden Effekt:

Definition 4.7. Kaskadierung (Cascading): Abgeleitet vom Begriff der Kaskade (einem stufenförmigen Wasserfall) beschreibt dies in der Informatik einen Effekt, der sich automatisch von einer oberen Ebene über alle darunterliegenden Ebenen fortpflanzt. Wenn die Eltern-Menge eingefroren wird, friert sie automatisch alle in ihr enthaltenen Kind-Strukturen ein. Diese frieren wiederum ihre eigenen Kinder ein, bis das Netz der verschachtelten Datenstrukturen versiegelt ist.

Da der `hashCode`-Getter bei allen strukturellen Elementen wie `RecursiveSet`, `Tuple` oder `RecursiveMap` exakt denselben Mechanismus beinhaltet, kaskadiert das Einfrieren durch den gesamten Baum hinab.

5 Vom Python-Werkzeug *PLY* zu *Lezer*

In den ursprünglichen Python-Implementierungen der Lehrveranstaltung werden zentrale Schritte der Sprachverarbeitung mit der Bibliothek *PLY* (Python Lex-Yacc) umgesetzt. *PLY* ist eine Parsergenerator-Bibliothek für Python, die sich an den klassischen Werkzeugen *Lex* und *Yacc* orientiert. Diese Werkzeuge werden häufig in Lehrumgebungen eingesetzt, um grundlegende Konzepte der Compilertechnik wie Tokenisierung und Parsing praktisch umzusetzen. Die Bibliothek wurde von David Beazley entwickelt und ist frei verfügbar [7].

Die grundlegende Architektur eines solchen Sprachprozessors besteht aus zwei zentralen Phasen: der lexikalischen Analyse und der syntaktischen Analyse. Zunächst zerlegt der Lexer den Eingabetext in eine Folge elementarer Token, anschließend analysiert der Parser diese Tokenfolge anhand einer kontextfreien Grammatik und konstruiert daraus eine syntaktische Struktur, etwa einen abstrakten Syntaxbaum (AST) [8, S.5ff].

PLY bildet diese beiden Komponenten explizit ab:

- **Lexikalische Analyse (Lexer):** Token werden über reguläre Ausdrücke definiert. Der Lexer erkennt daraus elementare Spracheinheiten wie Zahlen, Operatoren oder Schlüsselwörter.
- **Syntaxanalyse (Parser):** Die Grammatik der Sprache wird als Menge von Produktionsregeln formuliert. *PLY* generiert daraus einen Parser, der auf LR-Parsing-Techniken basiert. Diese Parsingverfahren lesen die Eingabe von links nach rechts und konstruieren eine rechtsableitende Analyse, wodurch auch komplexe Programmiersprachen effizient verarbeitet werden können.

Ein charakteristisches Merkmal von *PLY* ist, dass Token-Definitionen und Grammatikregeln direkt im Python-Code definiert werden. Produktionsregeln werden dabei als Funktionen formuliert, deren Docstrings die jeweilige Grammatikregel enthalten. Diese enge Integration erleichtert die prototypische Entwicklung von Parsern im Lehrkontext.

Während *PLY* somit eine klassische Compilerarchitektur innerhalb der Python-Umgebung realisiert, verfolgt das TypeScript-Ökosystem einen anderen Ansatz. Für editornahe Anwendungen und webbasierte Entwicklungsumgebungen wird häufig das Parser-Framework *Lezer* eingesetzt.

Lezer verfolgt das Ziel, syntaktische Analysen effizient und inkrementell durchzuführen. Im Gegensatz zu traditionellen Parsergeneratoren wird der Parser so konstruiert, dass er Änderungen am Quelltext lokal verarbeiten kann, ohne den gesamten Syntaxbaum neu berechnen zu müssen [9].

```

1 tokens = ('NUMBER', 'PLUS')
2
3 t_PLUS = r'\+'
4
5 def t_NUMBER(t):
6     r'\d+'
7     t.value = int(t.value)
8     return t

```

Listing 73: Ply-Grammatik in Python

Die Grammatik wird hierbei nicht mehr als Python-Code formuliert, sondern als deklarative Grammatikbeschreibung in einer eigenen Syntax definiert:

```

1 @top Expression { Add }
2
3 Add {
4     Expression "+" Expression
5 }

```

Listing 74: Lezer-Grammatik in TypeScript

Aus dieser Beschreibung generiert *Lezer* einen Parser, der direkt im TypeScript-Ökosystem verwendet werden kann.

Der zentrale Unterschied zwischen beiden Ansätzen liegt somit weniger in der zugrundeliegenden formalen Theorie. Beide basieren auf kontextfreien Grammatiken und klassischen Parsingverfahren, sie unterscheiden sich vielmehr in der Zielumgebung. Während *PLY* primär für klassische Compilerprototypen und Lehrzwecke in Python konzipiert wurde, ist *Lezer* auf moderne Entwicklungswerkzeuge und inkrementelle Analyse von Programmtexten ausgelegt.

Im Kontext dieser Arbeit dient *PLY* daher als Referenzimplementierung der ursprünglichen Python-Notebooks, während *Lezer* die Grundlage für die Umsetzung der gleichen Sprachverarbeitungsschritte im TypeScript-Ökosystem bildet.

5.1 Grundlagen von Lezer

Lezer ist ein Parser-Framework, das im Umfeld des CodeMirror-Projekts entwickelt wurde und speziell auf den Einsatz in modernen Entwicklungswerkzeugen ausgelegt ist [9]. Im Gegensatz zu vielen klassischen Parsergeneratoren steht bei *Lezer* nicht die einmalige Analyse eines vollständigen Programms im Vordergrund, sondern die effiziente Verarbeitung sich kontinuierlich ändernder Programmtexte.

Ein zentrales Ziel von *Lezer* besteht darin, die syntaktische Struktur eines Textes auch während der Bearbeitung im Editor schnell verfügbar zu machen. Funktionen wie Syntax-Highlighting, automatische Einrückung oder Fehlermarkierungen benötigen eine konsistente syntaktische Analyse des aktuellen Dokumentzustands. Da sich der Quelltext während der Bearbeitung ständig verändert, muss der Parser in der Lage sein, diese Änderungen effizient zu verarbeiten.

Inkrementelles Parsing

Eine zentrale Eigenschaft von *Lezer* ist das sogenannte *inkrementelle Parsing*. Dabei wird der Syntaxbaum eines Dokuments nicht bei jeder Änderung vollständig neu berechnet. Stattdessen werden nur jene Teile des Baums aktualisiert, die tatsächlich von der Änderung betroffen sind.

Dieser Ansatz reduziert den Rechenaufwand erheblich und ermöglicht eine nahezu verzögerungsfreie Aktualisierung der Syntaxstruktur im Editor. Besonders bei größeren Dokumenten oder komplexeren Grammatiken stellt dies einen entscheidenden Vorteil gegenüber klassischen Parserarchitekturen dar [10].

Baumrepräsentation der Syntax

Das Ergebnis der Analyse durch *Lezer* ist eine baumartige Repräsentation der syntaktischen Struktur des Eingabetextes. Diese Struktur enthält sämtliche syntaktischen Elemente, die durch die Grammatik definiert wurden.

Lezer erzeugt dabei zunächst einen sogenannten *Concrete Syntax Tree* (CST). Im Gegensatz zu einem abstrakten Syntaxbaum bleiben hier auch Zwischensymbole und strukturelle Details der Grammatik erhalten. Diese vollständige Darstellung der syntaktischen Struktur ist besonders für editornahe Funktionen relevant, da sie eine präzise Lokalisierung einzelner Sprachelemente ermöglicht.

Integration in das JavaScript- und TypeScript-Ökosystem

Ein weiterer wichtiger Aspekt von *Lezer* ist seine Integration in das JavaScript- und TypeScript-Ökosystem. Der generierte Parser kann direkt in Webanwendungen oder Entwicklungswerkzeuge (wie z.B. in Jupyter-Notebook) eingebunden werden.

Im Kontext dieser Arbeit wird *Lezer* verwendet, um die syntaktische Analyse der ursprünglichen Python-Lehrbeispiele in eine TypeScript-basierte Implementierung zu übertragen. Die Grammatik der Sprache bildet dabei die formale Grundlage für die Generierung des Parsers, während TypeScript für die Weiterverarbeitung der analysierten Strukturen eingesetzt wird.

Der nächste Abschnitt beschreibt daher die konkrete Definition der Grammatik, die als Grundlage für die spätere Parsergenerierung dient.

5.2 Grammatikdefinition in Lezer

Die syntaktische Struktur der zu analysierenden Sprache wird in *Lezer* durch eine deklarative Grammatikbeschreibung festgelegt. Diese Grammatik bildet die Grundlage für die spätere Generierung des Parsers und definiert, welche syntaktischen Strukturen innerhalb der Sprache zulässig sind.

Im Rahmen dieser Arbeit wurde die Grammatik direkt innerhalb der verwendeten Jupyter-Notebooks entwickelt. Dieses Vorgehen orientiert sich an der Struktur der ursprünglichen Lehrmaterialien, die ebenfalls notebookbasiert aufgebaut sind. Dadurch konnte die Grammatik schrittweise erweitert und unmittelbar anhand von Beispielausdrücken getestet werden.

Ein wesentliches Merkmal der vorliegenden Implementierung besteht darin, dass die Grammatik nicht in einer separaten Datei abgelegt wurde, sondern als mehrzeiliger String direkt im TypeScript-Code des Notebooks definiert ist. Die Grammatik wird somit als Datenobjekt innerhalb des Programms gehalten und kann ohne Zwischenschritt unmittelbar an die Parsergenerierung übergeben werden. Dieses Vorgehen vereinfacht den experimentellen Umgang mit der Grammatik, da Änderungen direkt an der Stringdefinition vorgenommen und sofort getestet werden können.

Struktur einer Lezer-Grammatik

Eine *Lezer*-Grammatik besteht aus mehreren Komponenten, die gemeinsam die syntaktische Struktur der Sprache beschreiben. Dazu gehören insbesondere:

- die Definition eines Startsymbols,
- Regeln zur Beschreibung komplexer syntaktischer Strukturen,
- sowie Token-Definitionen für elementare Sprachelemente.

Das folgende Beispiel zeigt eine vereinfachte Grammatikdefinition für arithmetische Ausdrücke, wie sie im Notebook als String festgehalten wird:

Die Variable `grammarDefinition` enthält die vollständige Grammatikbeschreibung als Zeichenkette. Durch diese Repräsentation bleibt die Grammatik eng mit dem restlichen Notebook-Code verknüpft. Insbesondere kann sie direkt an die Funktion zur Parsererzeugung übergeben werden, ohne dass ein externer Lade- oder Build-Schritt erforderlich ist.

Das Schlüsselwort `@top` definiert das Startsymbol der Grammatik. Dieses Symbol bildet den Einstiegspunkt der syntaktischen Analyse. Die Regel `Add` beschreibt eine mögliche Struktur eines Ausdrucks: Ein Ausdruck kann entweder aus zwei Teilausdrücken bestehen, die durch einen Additionsoperator verbunden sind, oder aus einem numerischen Wert.

```

1  const grammarDefinition = `
2      @top Expression { Add }
3
4      Add {
5          Expression "+" Expression |
6          Number
7      }
8
9      @tokens {
10         Number { ${[0-9]}+ }
11     }
12 `;

```

Listing 75: Umsetzung Lezer-Grammatik in TypeScript

Die Definition der Token erfolgt im Block `@tokens`. Hier werden reguläre Ausdrücke verwendet, um elementare Spracheinheiten zu erkennen. Diese Token bilden die Grundlage für die syntaktische Analyse, da sie die kleinsten Einheiten darstellen, aus denen komplexere Strukturen aufgebaut werden.

Iterative Entwicklung der Grammatik

Die Entwicklung der Grammatik erfolgte iterativ innerhalb der Notebook-Umgebung. Nach jeder Anpassung der Stringdefinition konnte der Parser neu generiert und direkt auf Beispielausdrücke angewendet werden. Dieses Vorgehen erleichtert insbesondere die schrittweise Erweiterung der Grammatik, da syntaktische Fehler oder Mehrdeutigkeiten frühzeitig erkannt werden können.

Darüber hinaus unterstützt die Notebook-Umgebung eine experimentelle Arbeitsweise, bei der einzelne Sprachkonstrukte isoliert getestet werden können. Dies ist insbesondere im Lehrkontext hilfreich, da die Auswirkungen einzelner Grammatikregeln unmittelbar sichtbar werden.

Die so definierte Grammatik bildet schließlich die Grundlage für die Generierung des Parsers, die im folgenden Abschnitt beschrieben wird.

5.3 Generierung des Parsers

Nachdem die Grammatik definiert wurde, kann daraus ein ausführbarer Parser erzeugt werden. In klassischen Projekten erfolgt dieser Schritt häufig über ein separates Generatorwerkzeug im Build-Prozess. Im Rahmen dieser Arbeit wurde die Parsergenerierung jedoch direkt innerhalb der Notebook-Umgebung durchgeführt.

Hierzu ist insbesondere das Paket `@lezer/generator` relevant. Nach der offiziellen Dokumentation übernimmt dieses Paket die Erzeugung der Parse-Tabellen

aus einer Grammatikbeschreibung. Üblicherweise wird es als Generatorwerkzeug im Build-Prozess eingesetzt; es kann jedoch auch direkt als JavaScript-Funktion verwendet werden [9]. Genau dieses Vorgehen wurde in der vorliegenden Arbeit genutzt.

Die grundlegende Erstellung des Parsers erfolgt über die Funktion `buildParser`, die aus `@lezer/generator` importiert wird. Diese Funktion nimmt die zuvor definierte Grammatik als Eingabe und erzeugt daraus eine Parserinstanz:

```
1 import { buildParser } from "@lezer/generator";  
2  
3 const parser = buildParser(grammarDefinition);
```

Listing 76: Erstellung des Parsers in TypeScript

Die Variable `grammarDefinition` enthält hierbei die deklarative Beschreibung der Grammatik. Auf Grundlage dieser Definition erzeugt *Lezer* die internen Parsingtabellen und Datenstrukturen, die für die syntaktische Analyse benötigt werden. Der so erzeugte Parser basiert zur Laufzeit auf dem *Lezer-LR*-System, während die erzeugten Syntaxbäume auf den gemeinsamen Baumstrukturen des *Lezer*-Ökosystems beruhen [9].

Parserinstanz

Das Ergebnis des Generierungsprozesses ist ein Parserobjekt, das anschließend zur Analyse von Eingabetexten verwendet werden kann. Der Parser verarbeitet einen Eingabestring und erzeugt daraus eine Baumstruktur, welche die syntaktische Struktur des Ausdrucks repräsentiert.

Diese Struktur ermöglicht es, einzelne Sprachelemente gezielt zu identifizieren und weiterzuverarbeiten. Insbesondere können nachgelagerte Verarbeitungsschritte auf bestimmte Knoten des Syntaxbaums zugreifen, um beispielsweise Ausdrücke auszuwerten oder Transformationen durchzuführen.

Integration in den Notebook-Workflow

Die Generierung des Parsers innerhalb des Notebooks bietet mehrere Vorteile für den Entwicklungsprozess. Änderungen an der Grammatik können unmittelbar getestet werden, da der Parser nach jeder Anpassung neu erzeugt werden kann. Dadurch entsteht ein iterativer Entwicklungsprozess, bei dem Grammatikdefinition und Parserverhalten eng miteinander verknüpft sind.

Dieses Vorgehen entspricht der didaktischen Struktur der ursprünglichen Lehrmaterialien, in denen Sprachkonstrukte schrittweise eingeführt und unmittelbar anhand von Beispielen überprüft werden.

Der generierte Parser bildet schließlich die Grundlage für die syntaktische Analyse innerhalb der TypeScript-Implementierung. Im folgenden Abschnitt wird beschrieben, wie der Parser in die eigentliche Programmlogik integriert wird und wie die erzeugten Syntaxbäume weiterverarbeitet werden.

5.4 Integration in die TypeScript-Implementierung

Die Generierung des Parsers stellt im Projekt keinen isolierten Endpunkt dar, sondern ist unmittelbar in die weitere TypeScript-Implementierung eingebettet. Der mit `buildParser(grammarDefinition)` erzeugte Parser wird direkt zur Analyse von Eingabestrings verwendet und bildet damit die Eingangsstufe für die weitere Verarbeitung der Sprachelemente.

Im konkreten Projekt greifen dabei mehrere Pakete des *Lezer*-Ökosystems zusammen. Das Paket `@lezer/generator` wird zur Erzeugung des Parsers aus der Grammatikdefinition verwendet. Das Paket `@lezer/common` stellt mit `TreeCursor` eine zentrale Schnittstelle für die Traversierung der erzeugten Baumstruktur bereit. Zusätzlich wird `@lezer/lr` für den LR-basierten Parser-Typ verwendet, sodass die Parserinstanz auch auf Typebene explizit beschrieben werden kann [9].

Parsing von Eingabetexten

Nach der Erzeugung der Parserinstanz kann ein Eingabetext über die Methode `parse` analysiert werden. Diese Methode ist nicht projektspezifisch implementiert, sondern Bestandteil der durch *Lezer* erzeugten Parserinstanz. Im Projekt wird sie über den Ausdruck `parser.parse(input)` aufgerufen. Das Ergebnis ist zunächst keine fachliche Datenstruktur der Anwendung, sondern eine vom Parser erzeugte syntaktische Baumrepräsentation. Diese Struktur enthält die durch die Grammatik erkannten Knoten und dient als unmittelbares Resultat der *Lezer*-Analyse.

Im Notebook wird dieser Schritt direkt mit der anschließenden Weiterverarbeitung verbunden. Ein typisches Muster besteht darin, den Eingabetext zunächst durch den generierten Parser analysieren zu lassen, anschließend einen Cursor auf dem erzeugten Baum zu erzeugen und diesen Baum danach in eine projektspezifische Datenstruktur zu überführen:

Dieses Beispiel verdeutlicht, dass im Projekt zwischen der von *Lezer* bereitgestellten Parsingfunktion und der eigenen Anwendungslogik unterschieden wird. Während `parser.parse(...)` die konkrete Syntax analysiert und einen Syntaxbaum erzeugt, übernimmt die Funktion `parseStmnt` die Kapselung dieses Schritts sowie die anschließende Transformation in eine typisierte interne Repräsentation.

```

1 import { buildParser } from "@lezer/generator";
2 import { TreeCursor } from "@lezer/common";
3
4 function parseStmtnt(input: string): Stmtnt {
5     const tree = parser.parse(input);
6     const ast = cstToAST(tree.cursor(), input, config);
7     if (ast instanceof AssignNode || ast instanceof ExprStmtNode) {
8         return ast;
9     }
10    throw new Error(`Expected a statement, got: ${ast.constructor.name}`);
11 }

```

Listing 77: ParseStmtnt Funktion in TypeScript

Transformation vom CST zum projektspezifischen AST

Für die eigentliche Programmlogik genügt die konkrete Syntaxstruktur allein nicht. Benötigt wird vielmehr eine Datenstruktur, die auf die Anforderungen der weiteren Verarbeitung abgestimmt ist. Aus diesem Grund wird der durch *Lezer* erzeugte Baum in einen projektspezifischen abstrakten Syntaxbaum (AST) überführt.

Im Projekt erfolgt diese Transformation über die Funktion `cstToAST`. Diese verarbeitet den vom Parser erzeugten Baum rekursiv anhand eines `TreeCursor`. Für jeden Knoten wird geprüft, ob eine zugehörige Transformationsregel definiert ist. Ist dies der Fall, wird aus den Kindknoten ein passender AST-Knoten konstruiert. Knoten, die lediglich syntaktische Zwischenstufen darstellen, können dabei ausgelassen oder automatisch reduziert werden. Die entsprechende Transformationslogik ist im Projekt generisch über `ParserConfig` und `TransformRule` organisiert, sodass die Abbildung vom konkreten Syntaxbaum auf den projektspezifischen AST regelbasiert erfolgt.

Dieses Vorgehen trennt die parsernahe Syntaxrepräsentation klar von der semantisch relevanten internen Modellierung. Die von *Lezer* erzeugte Struktur bleibt damit auf die Analyse des Eingabetexts beschränkt, während die eigentliche Programmlogik ausschließlich mit den definierten TypeScript-Typen und Klassen arbeitet.

Typsicherheit an der Schnittstelle

Ein weiterer Vorteil dieser Integration liegt in der klaren Typisierung der Schnittstelle zwischen Parser und Anwendungscode. Die Funktion `parseStmtnt` kapselt nicht nur den Aufruf des generierten Parsers, sondern überprüft zusätzlich, ob das Resultat tatsächlich die erwartete Form besitzt. Dadurch wird verhindert, dass nachgelagerte Verarbeitungsschritte mit ungeeigneten oder unerwarteten Knotenstrukturen arbeiten.

Gerade im Kontext dieser Arbeit ist dieser Schritt von Bedeutung, da die Portierung nach TypeScript nicht nur die syntaktische Analyse reproduzieren soll, sondern zugleich die Vorteile statischer Typisierung nutzbar macht. Die Integration des Parsers endet daher nicht beim Erzeugen eines Syntaxbaums, sondern umfasst auch dessen kontrollierte Überführung in wohldefinierte, typisierte Datenstrukturen.

Bedeutung für die Gesamtarchitektur

Die Einbettung des *Lezer*-Parsers in die TypeScript-Implementierung zeigt, dass der Parser im Projekt als Teil einer mehrstufigen Verarbeitungskette verstanden wird. Auf die formale Grammatikdefinition folgt die Parsererzeugung, anschließend die Analyse konkreter Eingaben durch den generierten Parser und schließlich die Transformation in eine interne Repräsentation, auf der Auswertung, Analyse oder weitere Übersetzungsschritte aufbauen können.

Damit übernimmt *Lezer* im Gesamtsystem die Aufgabe, die textuelle Eingabe in eine formal strukturierte Zwischenform zu überführen. Die fachliche Bedeutung der erkannten Konstrukte entsteht jedoch erst durch die nachgelagerte Abbildung auf die projektspezifischen AST-Klassen. Genau diese Trennung zwischen syntaktischer Analyse und semantischer Weiterverarbeitung macht die Integration in die TypeScript-Implementierung besonders anschlussfähig und wartbar.

6 Case Study: Umsetzung eines vollständigen Interpreters in TypeScript

In diesem Kapitel wird die praktische Umsetzung der zuvor entwickelten Konzepte anhand des letzten Notebooks (03-Interpreter.ipynb) aus Chapter-07 zur Interpreter-Realisierung dargestellt. Im Mittelpunkt steht nicht nur ein einzelner Ausdrucksparser, sondern eine vollständige Verarbeitungskette für eine kleine c-ähnliche Sprache: von der Grammatikdefinition über die Parsergenerierung und die Transformation des Concrete Syntax Tree (CST) in einen Abstract Syntax Tree (AST) bis hin zur eigentlichen Interpretation des Programms.

Die Fallstudie ist besonders geeignet, um die Unterschiede und Gemeinsamkeiten zwischen der ursprünglichen Python-basierten Lehrumgebung und der TypeScript-Variante sichtbar zu machen. Beide Implementierungen folgen denselben compilertechnischen Grundprinzipien, unterscheiden sich jedoch in der Modellierung der Datenstrukturen, im Zusammenspiel mit dem Typsystem und in der konkreten Werkzeugkette. Der grundsätzliche Ablauf entspricht den klassischen Phasen eines Sprachprozessors: syntaktische Analyse, interne Repräsentation und semantische Ausführung [8, S. 4 ff.].

6.1 Ziel und Aufbau des Notebooks

Das Notebook implementiert einen vollständigen Interpreter für eine kleine imperative Sprache mit Variablen, arithmetischen Ausdrücken, Bedingungen, Schleifen und elementaren Funktionsaufrufen. Die Verarbeitungskette besteht aus vier Schritten:

1. Definition der Sprache durch eine formale Grammatik,
2. Generierung eines Parsers mit Lezer,
3. Umwandlung des erzeugten CST in einen kompakteren AST,
4. rekursive Interpretation des AST.

Diese Struktur entspricht sowohl in der Python- als auch in der TypeScript-Implementierung dem klassischen Aufbau eines Interpreters. Während die Python-Variante typischerweise auf PLY basiert und Lexer sowie Parser explizit trennt, wird in der TypeScript-Version mit Lezer eine integrierte, deklarative Grammatik verwendet. Unabhängig vom eingesetzten Werkzeug bleibt die zugrunde liegende Architektur identisch. Eine formale Grammatik beschreibt die syntaktisch zulässigen Programme, der Parser konstruiert daraus eine Baumdarstellung, und ein

nachgelagerter Verarbeitungsschritt interpretiert diese Repräsentation semantisch [8, S.42ff.].

6.2 Sprachdefinition im Vergleich: Python (PLY) vs. TypeScript (Lezer)

In der ursprünglichen Python-Implementierung wird die Sprache mithilfe von *PLY* (Python Lex-Yacc) definiert. Dabei erfolgt eine klassische Trennung zwischen lexikalischer und syntaktischer Analyse, wie sie in der Compiler-Technik etabliert ist [8, S. 5ff]. Die Tokenisierung wird durch reguläre Ausdrücke beschrieben, während die Grammatikregeln in Form von Python-Funktionen implementiert werden. Dieses Vorgehen orientiert sich eng an traditionellen Werkzeugen wie *Lex* und *Yacc* und spiegelt den zweiphasigen Aufbau eines Compilers wieder.

Dieses Vorgehen führt zu einer engen Verzahnung von Grammatik und Programmlogik. Grammatikregeln sind implizit in Python-Code eingebettet, wodurch die formale Struktur der Sprache weniger explizit sichtbar ist. Gleichzeitig ermöglicht dieser Ansatz eine flexible Integration semantischer Aktionen direkt in den Parser.

Im Gegensatz dazu wird die Sprache in der TypeScript-Implementierung mit *Lezer* deklarativ als Grammatik spezifiziert. Die Grammatik wird dabei als strukturierter String formuliert und anschließend automatisch in einen Parser übersetzt. Dieses Vorgehen entspricht stärker der formalen Definition kontextfreier Grammatiken, wie sie in der Theorie formaler Sprachen üblich sind [11, S. 171ff].

Ein wesentlicher Unterschied zwischen beiden Ansätzen liegt somit im Abstraktionsniveau: Während *PLY* eine imperativ definierte Kombination aus Lexer- und Parserlogik darstellt, beschreibt *Lezer* die Sprache deklarativ auf Basis einer expliziten Grammatik. Diese deklarative Form erleichtert insbesondere die Analyse, Wartbarkeit und Weiterverarbeitung der Grammatik, da die Struktur der Sprache klar getrennt von ihrer Verarbeitung beschrieben wird.

Darüber hinaus unterstützt *Lezer* moderne Anforderungen wie inkrementelles Parsing, das insbesondere für editornähe Anwendungen relevant ist. Der Parser kann dabei Änderungen effizient verarbeiten, ohne die gesamte Eingabe neu analysieren zu müssen. Dies stellt einen wesentlichen Vorteil gegenüber klassischen Parsergeneratoren dar, die typischerweise eine vollständige Neuverarbeitung der Eingabe voraussetzen.

Zusammenfassend lässt sich festhalten, dass *PLY* einen eher traditionellen, eng an der Implementierung orientierten Ansatz verfolgt, während *Lezer* eine stärker formal orientierte und deklarative Beschreibung der Sprache ermöglicht. Diese Unterschiede sind insbesondere im Kontext der Übersetzung von Python

```

1 import ply.lex as lex
2 tokens = [
3     'NUMBER',      # r'0|[1-9][0-9]*'
4     'IDENTIFIER',  # r'[a-zA-Z][a-zA-Z0-9_]*'
5     'ASSIGN',      # r':='
6     'EQ',          # r'=='
7     'NE',          # r'!='
8     'LE',          # r'<='
9     'GE',          # r'>='
10    'IF',           # see below
11    'WHILE'         # see below
12 ]
13 def t_COMMENT(t):
14     r'(/\*(.|\n)*?\/|\/\/.*'
15     t.lexer.lineno += t.value.count('\n')
16 def t_NUMBER(t):
17     r'0|[1-9][0-9]*'
18     t.value = int(t.value)
19     return t
20 t_ASSIGN = r':='
21 t_EQ     = r'=='
22 t_NE     = r'!='
23 t_LE     = r'<='
24 t_GE     = r'>='
25 Keywords = { 'if': 'IF', 'while': 'WHILE' }
26 def t_IDENTIFIER(t):
27     r'[a-zA-Z][a-zA-Z0-9_]*'
28     t.type = Keywords.get(t.value, 'IDENTIFIER')
29     return t
30 literals = ['+', '-', '*', '/', '%', '(', ')', '{', '}', ';', '<', '>', ',']
31 t_ignore = ' \t\r'
32 def t_newline(t):
33     r'\n'
34     t.lexer.lineno += 1
35     return
36 def find_column(token):
37     program = token.lexer.lexdata # the complete string given to the scanner
38     line_start = program.rfind('\n', 0, token.lexpos)
39     return token.lexpos - line_start
40 def t_error(t):
41     column = find_column(t)
42     print(f"Illegal character '{t.value[0]}' in line {t.lineno}, column {column}.")
43     t.lexer.skip(1)
44 lexer = lex.lex()

```

Listing 78: Ply-Grammatik

```

1 import { buildParser } from "@lezer/generator";
2 const grammarString = `
3   @top Script { Program }
4   @tokens {
5     Identifier { ${[a-zA-Z]} ${[a-zA-Z0-9_]}* }
6     Number      { "0" | ${[1-9]} ${[0-9]}* }
7     "+" "-" "*" "/" "%"
8     ":@" "==" "!=" "<=" ">=" "<" ">"
9     "(" ")" "{" "}"
10    ";" " ",
11    space { ${[ \t\n\r]}+ }
12    LineComment { "//" ![\n]* }
13    BlockComment { "/*" ( ![*] | "*" + ![*/] )* "*" + "/" }
14    @precedence { LineComment, BlockComment, "/" }
15  }
16  @skip { space | LineComment | BlockComment }
17  IF { @specialize<Identifier, "if"> }
18  WHILE { @specialize<Identifier, "while"> }
19  Program {
20    Stmtnt Program |
21    ""
22  }
23  Stmtnt {
24    IF "(" BoolExpr ")" Stmtnt |
25    WHILE "(" BoolExpr ")" Stmtnt |
26    "{" Program "}" |
27    Identifier ":@" Expr ";" |
28    Expr ";"
29  }
30  BoolExpr {
31    Expr "==" Expr |
32    Expr "!=" Expr |
33    Expr "<=" Expr |
34    Expr ">=" Expr |
35    Expr "<" Expr |
36    Expr ">" Expr
37  }
38  Expr {
39    Expr "+" Product |
40    Expr "-" Product |
41    Product
42  }
`

```

Listing 79: Lezer-Grammatik-Teil1

```

1  `
2      Product {
3          Product "*" Factor |
4          Product "/" Factor |
5          Product "%" Factor |
6          Factor
7      }
8      Factor {
9          "(" Expr ")"      |
10         Number            |
11         Identifier        |
12         Identifier "(" ExprList ")"
13     }
14     ExprList {
15         NeExprList |
16         ""
17     }
18     NeExprList {
19         Expr |
20         Expr "," NeExprList
21     }
22 `;
23 const parser = buildParser(grammarString);

```

Listing 80: Lezer-Grammatik-Teil2

nach TypeScript relevant, da sie unterschiedliche Möglichkeiten zur Modellierung und Weiterverarbeitung von Sprachstrukturen bieten.

6.3 Transformation vom CST zum AST

Ein wesentlicher Unterschied zwischen beiden Implementierungen zeigt sich unmittelbar nach dem Parsing. In der Python-Version wird der AST bereits während der Anwendung der Grammatikregeln erzeugt. Die semantischen Aktionen der PLY-Produktionen konstruieren direkt die interne Repräsentation, sodass keine separate Transformationsphase zwischen Parsebaum und abstrakter Syntax mehr erforderlich ist. Die Grammatik und die Konstruktion der Zielstruktur sind damit eng miteinander verbunden.

Die TypeScript-Variante trennt diese Schritte klarer. Lezer erzeugt zunächst einen CST, der anschließend durch eine eigene Transformationsfunktion in einen AST überführt wird. Dadurch wird die klassische Trennung zwischen konkreter und abstrakter Syntax explizit sichtbar [8, S. 42ff.].

Die Python-Variante erzeugt ihre Zielstruktur somit unmittelbar innerhalb der Grammatikregeln, während die TypeScript-Version einen expliziten Zwischenschritt einführt. Diese zusätzliche Transformationsphase erhöht die Modularität der Implementierung und trennt die formale Beschreibung der Sprache deutlicher von ihrer internen Repräsentation.

6.4 Darstellung rekursiver Listenstrukturen

Auch bei der Modellierung rekursiver Listenstrukturen folgen beide Notebooks demselben Grundprinzip. Sowohl Programmlisten als auch Argumentlisten werden als Punktlisten dargestellt. In der Python-Version geschieht dies direkt in den PLY-Regeln.

In der TypeScript-Version bleibt diese Listenstruktur ebenfalls erhalten, wird jedoch explizit durch eine eigene Ausführungsfunktion verarbeitet. Die rekursive Form wird damit nicht nur beim Aufbau des AST, sondern auch bei seiner semantischen Verarbeitung sichtbar.

Die Gemeinsamkeit beider Varianten liegt darin, dass die rekursive Struktur der Grammatik unmittelbar in die interne Repräsentation übernommen wird. Der Unterschied besteht vor allem darin, dass TypeScript diese Listenstruktur zusätzlich durch Hilfsfunktionen systematischer kapselt.

```

1 start = 'program'
2 def p_program_lambda(p):
3     "program : "
4     p[0] = ('.',)
5 def p_program_more(p):
6     "program : stmtnt program"
7     p[0] = ('.', p[1]) + p[2][1:]
8 def p_stmtnt_if(p):
9     "stmtnt : IF '(' bool_expr ')' stmtnt"
10    p[0] = ('if', p[3], p[5])
11
12 def p_stmtnt_while(p):
13     "stmtnt : WHILE '(' bool_expr ')' stmtnt"
14    p[0] = ('while', p[3], p[5])
15
16 def p_stmtnt_block(p):
17     "stmtnt : '{' program '}'"
18    p[0] = p[2]
19
20 def p_stmtnt_assign(p):
21     "stmtnt : IDENTIFIER ASSIGN expr ';' "
22    p[0] = (':=', p[1], p[3])
23
24 def p_stmtnt_expr(p):
25     "stmtnt : expr ';' "
26    p[0] = ('expr', p[1])
27
28 def p_bool_expr_eq(p):
29     "bool_expr : expr EQ expr"
30    p[0] = ('==', p[1], p[3])
31
32 def p_bool_expr_ne(p):
33     "bool_expr : expr NE expr"
34    p[0] = ('!=', p[1], p[3])
35
36 def p_bool_expr_le(p):
37     "bool_expr : expr LE expr"
38    p[0] = ('<=', p[1], p[3])
39

```

Listing 81: Direkte AST-Konstruktion in PLY Teil1

```

1 def p_bool_expr_ge(p):
2     "bool_expr : expr GE expr"
3     p[0] = ('>=', p[1], p[3])
4
5 def p_bool_expr_lt(p):
6     "bool_expr : expr '<' expr"
7     p[0] = ('<', p[1], p[3])
8
9 def p_bool_expr_gt(p):
10    "bool_expr : expr '>' expr"
11    p[0] = ('>', p[1], p[3])
12
13 def p_expr_plus(p):
14    "expr : expr '+' product"
15    p[0] = ('+', p[1], p[3])
16
17 def p_expr_minus(p):
18    "expr : expr '-' product"
19    p[0] = ('-', p[1], p[3])
20
21 def p_expr_product(p):
22    "expr : product"
23    p[0] = p[1]
24
25 def p_product_times(p):
26    "product : product '*' factor"
27    p[0] = ('*', p[1], p[3])
28
29 def p_product_divide(p):
30    "product : product '/' factor"
31    p[0] = ('/', p[1], p[3])
32
33 def p_product_modulo(p):
34    "product : product '%' factor"
35    p[0] = ('%', p[1], p[3])

```

Listing 82: Direkte AST-Konstruktion in PLY Teil2

```

1 def p_product_factor(p):
2     "product : factor"
3     p[0] = p[1]
4
5 def p_factor_paren(p):
6     "factor : '(' expr '"
7     p[0] = p[2]
8
9 def p_factor_number(p):
10    "factor : NUMBER"
11    p[0] = p[1]
12
13 def p_factor_identifier(p):
14    "factor : IDENTIFIER"
15    p[0] = p[1]
16
17 def p_factor_fct_call(p):
18    "factor : IDENTIFIER '(' expr_list '"
19    p[0] = ('call', p[1]) + p[3][1:]
20
21 def p_expr_list_empty(p):
22    "expr_list : "
23    p[0] = ('.',)
24
25 def p_expr_more(p):
26    "expr_list : ne_expr_list"
27    p[0] = p[1]
28
29 def p_ne_expr_list_one(p):
30    "ne_expr_list : expr"
31    p[0] = ('.', p[1])
32
33 def p_ne_expr_list_more(p):
34    "ne_expr_list : expr ',' ne_expr_list"
35    p[0] = ('.', p[1]) + p[3][1:]
36 def p_error(t):
37     column = find_column(t)
38     if t:
39         print(f'Syntax error at token "{t.value}" in line {t.lineno}, column {column}.')
40     else:
41         print('Syntax error at end of input.')
42 parser = yacc.yacc(write_tables=False, debug=True)

```

Listing 83: Direkte AST-Konstruktion in PLY Teil3

```

1  import { cst2ast } from './CST2AST';
2
3  const myOperators = [
4      "IF", "WHILE", ":",
5      "+", "-", "*", "/", "%",
6      "==", "!=", "<", ">", "<=", ">="
7  ];
8
9  const myListVars = ["Program", "NeExprList"];
10
11 function parse(fileName: string): AST {
12     const source = readFileSync(fileName, "utf8");
13     const tree = parser.parse(source);
14     return cst2ast(tree.cursor(), source, myOperators, myListVars);
15 }

```

Listing 84: CST-zu-AST-Transformation in TypeScript

```

1  def p_program_lambda(p):
2      "program : "
3      p[0] = ('.',)
4
5  def p_program_more(p):
6      "program : stmt program"
7      p[0] = ('.', p[1]) + p[2][1:]
8
9  def p_expr_list_empty(p):
10     "expr_list : "
11     p[0] = ('.',)
12
13  def p_expr_more(p):
14     "expr_list : ne_expr_list"
15     p[0] = p[1]
16
17  def p_ne_expr_list_one(p):
18     "ne_expr_list : expr"
19     p[0] = ('.', p[1])
20
21  def p_ne_expr_list_more(p):
22     "ne_expr_list : expr ',' ne_expr_list"
23     p[0] = ('.', p[1]) + p[3][1:]

```

Listing 85: Rekursive Listen in der Python-Version

```

1 executeList = (list: AST, env: Environment) => {
2     if (list === "") return;
3
4     if (Array.isArray(list) && list[0] === '.') {
5         const head = list[1];
6         const tail = list[2];
7
8         if (head !== undefined && tail !== undefined) {
9             execute(head, env);
10            executeList(tail, env);
11        }
12    }
13 };

```

Listing 86: Verarbeitung rekursiver Listen in TypeScript

6.5 Typisierung und interne AST-Repräsentation

Ein besonders markanter Unterschied zwischen beiden Implementierungen liegt in der Darstellung des AST. In Python bleibt die Baumstruktur implizit durch Tupelkonventionen beschrieben. Welche Form ein Knoten besitzt, ergibt sich aus dem verwendeten Tag und der Position seiner Operanden. In TypeScript wird dieselbe Struktur dagegen durch einen expliziten Typ beschrieben.

```

1 type Operator = string;
2 type AST = string | number |
3     [Operator, AST] |
4     [Operator, AST, AST] |
5     [Operator, AST, AST, AST] |
6     [Operator, AST, AST, AST, AST];

```

Listing 87: AST-Typdefinition in TypeScript

Diese Typdefinition dokumentiert die rekursive Struktur des AST unmittelbar im Programmtext. Während in Python nur informell klar ist, dass etwa `('+', lhs, rhs)` einen binären Ausdruck repräsentiert, wird in TypeScript explizit festgelegt, dass AST-Knoten aus Zahlen, Bezeichnern oder Operator-Tupeln verschiedener Stelligkeit bestehen. Dadurch wird die Baumstruktur besser dokumentiert und teilweise statisch überprüfbar.

6.6 Interpreterarchitektur im Vergleich

Die Grundstruktur des Interpreters bleibt in beiden Notebooks gleich. Sowohl in Python als auch in TypeScript erfolgt eine Trennung in Anweisungsausführung, arithmetische Auswertung und boolesche Auswertung. In der Python-Version wird dies durch Pattern Matching über Tupel realisiert.

```
1 def execute(stmtnt: NestedTuple, Values: dict[str, Number]) -> None:
2     match stmtnt:
3         case ('.', *SL):
4             execute_tuple(tuple(SL), Values)
5         case (':=', var, value):
6             Values[var] = evaluate(value, Values)
7         case ('expr', expr):
8             evaluate(expr, Values)
9         case ('if', test, stmtnt):
10            if evaluate_bool(test, Values):
11                execute(stmtnt, Values)
12        case ('while', test, stmtnt):
13            while evaluate_bool(test, Values):
14                execute(stmtnt, Values)
15        case _:
16            assert False, f'{stmtnt} unexpected'
```

Listing 88: Ausführung von Anweisungen in Python

Die TypeScript-Version folgt derselben semantischen Idee, verwendet aber explizite Fallunterscheidungen über Knotentags.

Die semantische Struktur bleibt also erhalten, obwohl sich die sprachspezifischen Mittel unterscheiden. Python nutzt Tupel und Pattern Matching, TypeScript verwendet Array-Prüfungen und String-Tags.

6.7 Auswertung arithmetischer Ausdrücke

Auch bei der Auswertung arithmetischer Ausdrücke zeigen sich deutliche strukturelle Parallelen. In Python wird die Semantik direkt durch rekursive Fallunterscheidung über die Tupelform des Knotens beschrieben.

Die TypeScript-Version bildet dieselbe rekursive Struktur nach, ergänzt jedoch explizite Prüfungen auf undefinierte Variablen sowie Fehlerbehandlung bei Division durch Null.

Die prinzipielle Auswertungslogik bleibt damit erhalten. Auffällig ist jedoch, dass die TypeScript-Version die Division mit `Math.floor(l / r)` als ganzzahlige Division realisiert, während in Python direkt der Operator `/` verwendet wird. Damit

```

1 execute = (node: AST, env: Environment): void => {
2     if (!Array.isArray(node)) return;
3     const tag = node[0];
4
5     if (tag === '.') {
6         executeList(node, env);
7         return;
8     }
9
10    if (tag === "block") {
11        const content = node[1];
12        if (content !== undefined) {
13            executeList(content, env);
14        }
15    }
16    else if (tag === ":=") {
17        const name = node[1];
18        const expr = node[2];
19        if (typeof name === "string" && expr !== undefined) {
20            env.set(name, evaluate(expr, env));
21        }
22    }
23    else if (tag === "IF") {
24        const cond = node[1];
25        const body = node[2];
26        if (cond !== undefined && body !== undefined) {
27            if (evaluateBool(cond, env)) execute(body, env);
28        }
29    }
30    else if (tag === "WHILE") {
31        const cond = node[1];
32        const body = node[2];
33        if (cond !== undefined && body !== undefined) {
34            while (evaluateBool(cond, env)) execute(body, env);
35        }
36    }
37    else if (tag === "Call") {
38        evaluate(node, env);
39    }
40 };

```

Listing 89: Ausführung von Anweisungen in TypeScript

```

1 def evaluate(expr: NestedTuple, Values: dict[str, Number]) -> Number:
2     match expr:
3         case int():
4             return expr
5         case str():
6             return Values[expr]
7         case ('call', 'read'):
8             return int(input('Please enter a natural number: '))
9         case ('call', 'print', expr):
10            print(evaluate(expr, Values))
11            return 0;
12         case ('+', lhs, rhs):
13             return evaluate(lhs, Values) + evaluate(rhs, Values)
14         case ('-', lhs, rhs):
15             return evaluate(lhs, Values) - evaluate(rhs, Values)
16         case ('*', lhs, rhs):
17             return evaluate(lhs, Values) * evaluate(rhs, Values)
18         case ('/', lhs, rhs):
19             return evaluate(lhs, Values) / evaluate(rhs, Values)
20         case ('%', lhs, rhs):
21             return evaluate(lhs, Values) % evaluate(rhs, Values)
22         case _:
23             assert False, f'{expr} unexpected'

```

Listing 90: Arithmetische Auswertung in Python

```

1  evaluate = (node: AST, env: Environment): number => {
2      if (typeof node === "number") return node;
3      if (typeof node === "string") {
4          const val = env.get(node);
5          if (val === undefined) throw new Error(`Undefined var ${node}`);
6          return val;
7      }
8
9      if (Array.isArray(node)) {
10         const tag = node[0];
11
12         if (["+","-","*","/","%"].includes(tag)) {
13             const leftNode = node[1];
14             const rightNode = node[2];
15
16             if (leftNode !== undefined && rightNode !== undefined) {
17                 const l = evaluate(leftNode, env);
18                 const r = evaluate(rightNode, env);
19                 switch(tag) {
20                     case "+": return l + r;
21                     case "-": return l - r;
22                     case "*": return l * r;
23                     case "/":
24                         if (r === 0) throw new Error("Division by zero");
25                         return Math.floor(l / r);
26                     case "%":
27                         if (r === 0) throw new Error("Modulo by zero");
28                         return l % r;
29                 }
30             }
31         }
32         if (tag === "Call") {
33             const fn = node[1];
34             const argsNode = node[2];
35
36             if (fn === "print" && argsNode !== undefined) {
37                 if (Array.isArray(argsNode) && argsNode[0] === '.') {
38                     const actualArg = argsNode[1];
39                     if (actualArg !== undefined) {
40                         console.log(">> STDOUT:", evaluate(actualArg, env));
41                     }
42                 }
43             }
44         }
45     }
46 }

```

Listing 91: Arithmetische Auswertung in TypeScript Teil 1

zeigt sich, dass eine Portierung nicht nur strukturelle, sondern auch semantische Detailentscheidungen umfasst.

6.8 Auswertung boolescher Bedingungen

Die boolesche Auswertung ist in beiden Implementierungen analog aufgebaut. Vergleichsoperatoren werden rekursiv ausgewertet und liefern einen Wahrheitswert zurück.

```
1 def evaluate_bool(expr: NestedTuple, Values: dict[str, Number]) -> bool:
2     match expr:
3         case ('==', lhs, rhs):
4             return evaluate(lhs, Values) == evaluate(rhs, Values)
5         case ('!=', lhs, rhs):
6             return evaluate(lhs, Values) != evaluate(rhs, Values)
7         case ('<=', lhs, rhs):
8             return evaluate(lhs, Values) <= evaluate(rhs, Values)
9         case ('>=', lhs, rhs):
10            return evaluate(lhs, Values) >= evaluate(rhs, Values)
11        case ('<', lhs, rhs):
12            return evaluate(lhs, Values) < evaluate(rhs, Values)
13        case ('>', lhs, rhs):
14            return evaluate(lhs, Values) > evaluate(rhs, Values)
15        case _:
16            assert False, f'{expr} unexpected'
```

Listing 92: Boolesche Auswertung in Python

Hier zeigt sich besonders deutlich, dass die eigentliche Interpretersemantik weitgehend unverändert bleibt. Unterschiede ergeben sich vor allem aus der konkreten Form der AST-Repräsentation und aus den sprachspezifischen Mitteln der Fallunterscheidung.

6.9 Umgebungsmodell und Programmausführung

Der Programmlauf basiert in beiden Notebooks auf einer Variablenumgebung, die Bezeichner auf numerische Werte abbildet. In Python wird hierfür ein Dictionary verwendet, in TypeScript eine `Map<string, number>`.

Gerade an dieser Stelle wird ein praktischer Unterschied sichtbar: Während die Python-Version interaktive Eingaben direkt über `input(...)` verarbeitet, simuliert die TypeScript-Version Eingaben über einen vorbereiteten `inputStream`. Dies erleichtert insbesondere in Notebook-Umgebungen reproduzierbare Testläufe.

```

1 evaluateBool = (node: AST, env: Environment): boolean => {
2     if (Array.isArray(node)) {
3         const tag = node[0];
4         if (["==", "!=", "<", ">", "<=", ">="].includes(tag)) {
5             const leftNode = node[1];
6             const rightNode = node[2];
7
8             if (leftNode !== undefined && rightNode !== undefined) {
9                 const l = evaluate(leftNode, env);
10                const r = evaluate(rightNode, env);
11                switch(tag) {
12                    case "==": return l == r;
13                    case "!=": return l != r;
14                    case "<": return l < r;
15                    case ">": return l > r;
16                    case "<=": return l <= r;
17                    case ">=": return l >= r;
18                }
19            }
20        }
21    }
22    throw new Error("Invalid boolean expression");
23 };

```

Listing 93: Boolesche Auswertung in TypeScript

```

1 def main(file):
2     with open(file, 'r') as handle:
3         program = handle.read()
4         stmtnt = yacc.parse(program)
5         print(stmtnt)
6         Values = {}
7         execute(stmtnt, Values)

```

Listing 94: Programmausführung in Python

```

1  type Environment = Map<string, number>;
2  let inputStream: string[] = [];
3
4  function runProgram(fileName: string, inputs: string[] = []) {
5      console.log(`\n--- Executing File: ${fileName} ---`);
6      console.log(`--- Inputs: [${inputs.join(", ")}] ---`);
7      inputStream = [...inputs];
8      console.log(inputStream);
9      try {
10         const ast = parse(fileName);
11         execute(ast, new Map());
12     } catch (e) { console.error(e); }
13 }

```

Listing 95: Programmausführung in TypeScript

6.10 Ergebnisse der Case Study

Die Gegenüberstellung der restlichen Notebook-Bestandteile bestätigt die zentrale Beobachtung dieser Fallstudie: Die TypeScript-Version übernimmt die grundlegende Interpreterarchitektur des Python-Originals, modelliert die einzelnen Verarbeitungsschritte jedoch expliziter und modularer. Besonders deutlich wird dies an der Trennung von CST und AST, an der typisierten Beschreibung der Baumstruktur sowie an der systematischeren Behandlung rekursiver Listen und Laufzeitfehler.

Gleichzeitig zeigt der Vergleich der konkreten Codeausschnitte, dass die Portierung nicht nur eine Übertragung syntaktischer Konstrukte darstellt. Vielmehr werden bei der Umsetzung in TypeScript Entscheidungen über Abstraktionsniveau, Fehlerbehandlung und teilweise auch über semantische Details getroffen. Die TypeScript-Variante erscheint dadurch als stärker strukturierte und typisierte Rekonstruktion derselben interpretertechnischen Grundidee.

7 Fazit

Ziel dieser Arbeit war die Übertragung der interaktiven Python-basierten Lehrnotebooks der Vorlesung „Theoretische Informatik III“ in das TypeScript-Ökosystem. Dabei wurde bewusst nicht nur eine syntaktische Übersetzung angestrebt, sondern eine strukturelle Neuausrichtung der zugrunde liegenden Architektur. Im Zentrum stand die Frage, inwiefern sich Konzepte formaler Sprachen durch statische Typisierung präziser modellieren und zuverlässiger umsetzen lassen.

Die Ergebnisse der Arbeit zeigen deutlich, dass TypeScript durch sein statisches Typsystem einen wesentlichen Mehrwert für die Modellierung komplexer Datenstrukturen bietet. Während Python viele Fehler erst zur Laufzeit sichtbar macht, ermöglicht TypeScript eine frühzeitige Validierung von Invarianten bereits während der Entwicklung. Insbesondere bei verschachtelten Strukturen wie Syntaxbäumen oder Zustandsmengen trägt dies maßgeblich zur Robustheit der Implementierung bei. Damit wird ein zentraler Vorteil statischer Typisierung bestätigt, nämlich die Vorverlagerung von Fehlererkennung in die Entwicklungsphase [8, S. 371ff].

Ein zentrales Ergebnis der Arbeit ist die Erkenntnis, dass eine naive Portierung nicht ausreicht. Unterschiede in der Laufzeitsemantik, wie etwa das referenzbasierte Verhalten von Mengen in JavaScript, erfordern eigenständige architektonische Lösungen. Die entwickelte Datenstruktur `RecursiveSet` zeigt exemplarisch, wie durch gezielte Implementierungsentscheidungen mathematische Konzepte wie Wertgleichheit und Immutabilität in TypeScript nachgebildet werden können. Gleichzeitig verdeutlicht sie, dass Typsysteme allein nicht alle semantischen Unterschiede zwischen Programmiersprachen überbrücken können, sondern durch geeignete Datenstrukturen ergänzt werden müssen.

Auch im Bereich der Sprachverarbeitung konnte gezeigt werden, dass die grundlegenden Prinzipien der Compilertechnik unabhängig von der Zielplattform bestehen bleiben. Die Transformation von der lexikalischen Analyse über die syntaktische Analyse bis hin zur semantischen Interpretation folgt sowohl in der Python- als auch in der TypeScript-Implementierung denselben theoretischen Grundlagen [8, S. 4ff]. Der Einsatz von Lezer als Parsergenerator verdeutlicht jedoch, wie sich diese klassischen Konzepte an moderne Anforderungen wie inkrementelles Parsing und editornahe Anwendungen anpassen lassen.

Die Case Study zur Implementierung eines vollständigen Interpreters zeigt schließlich, dass sich die gesamte Verarbeitungskette – von der Grammatikdefinition bis zur Ausführung – erfolgreich in TypeScript abbilden lässt. Dabei wird deutlich, dass die statische Typisierung nicht nur der Fehlervermeidung dient, sondern auch die Struktur und Lesbarkeit des Codes verbessert. Gleichzeitig bleibt

ein gewisser Trade-off bestehen: Eine vollständig präzise Typmodellierung kann zu erhöhter Komplexität führen und erfordert bewusste Entwurfsentscheidungen zwischen Strenge und Flexibilität.

Zusammenfassend lässt sich festhalten, dass die Übersetzung der Lehrmaterialien nach TypeScript nicht nur technisch möglich, sondern auch didaktisch sinnvoll ist. Die stärkere Typisierung unterstützt das Verständnis formaler Strukturen, zwingt zur expliziten Modellierung von Invarianten und reduziert typische Fehlerquellen dynamischer Systeme. Gleichzeitig zeigt die Arbeit, dass eine erfolgreiche Migration stets eine konzeptionelle Anpassung erfordert und nicht auf eine reine Syntaxübertragung reduziert werden kann.

Die entwickelten Ansätze bilden damit eine solide Grundlage für zukünftige Erweiterungen, etwa die weitergehende Typisierung von AST-Strukturen oder die Integration zusätzlicher Analyse- und Optimierungsschritte innerhalb der Interpreterpipeline.

8 Literaturverzeichnis

- [1] IEEE Computer Society. *IEEE Standard for Floating-Point Arithmetic*. IEEE Std 754-2019 (Revision of IEEE 754-2008). IEEE, Juli 2019. URL: https://www-users.cse.umn.edu/~vinals/tspot_files/phys4041/2020/IEEE%20Standard%20754-2019.pdf.
- [2] Georg Cantor. „Beiträge zur Begründung der transfiniten Mengenlehre“. In: *Mathematische Annalen* 46 (1895), S. 481–512. URL: <http://eudml.org/doc/157768>.
- [3] Austin Appleby. *MurmurHash3*. <https://chromium.googlesource.com/external/smhasher/+58dd8869da8c95f5c26ec70a6cdd243a7647c8fc/MurmurHash3.cpp>. C++ Referenzimplementierung. Chromium-Mirror, abgerufen am 08.04.2026. 2011.
- [4] Richard Fabian. *Data-Oriented Design: Software Engineering for Limited Resources and Short Schedules*. Frei zugängliche PDF-Version. Richard Fabian, 2018. ISBN: 978-1916478701. URL: <https://dataorienteddesign.com/dodbook.pdf>.
- [5] Thomas H. Cormen u. a. *Introduction to Algorithms*. 4. Aufl. Abgerufen am 08.04.2026. MIT Press, 2022. URL: <https://iitdh.ac.in/~konjengbam.and/courses/daa/CLRS4e2022.pdf>.
- [6] Pedro Celis. *Robin Hood Hashing*. Techn. Ber. CS-86-14. Frei zugängliches PDF. University of Waterloo, Department of Computer Science, 1986. URL: <https://cs.uwaterloo.ca/research/tr/1986/CS-86-14.pdf>.
- [7] David Beazley. *PLY (Python Lex-Yacc)*. 2026. URL: <https://www.dabeaz.com/ply/> (besucht am 08.02.2026).
- [8] Alfred V. Aho u. a. *Compilers: Principles, Techniques, and Tools*. 2. Aufl. Addison-Wesley, 2006. ISBN: 978-0321486813.
- [9] *Lezer Documentation*. CodeMirror / Lezer Project. 2026. URL: <https://lezer.codemirror.net/docs/> (besucht am 08.02.2026).
- [10] Marijn Haverbeke. *Lezer*. 3. Sep. 2019. URL: <https://marijnhaverbeke.nl/blog/lezer.html> (besucht am 08.02.2026).
- [11] John E. Hopcroft, Rajeev Motwani und Jeffrey D. Ullman. *Introduction to Automata Theory, Languages, and Computation*. 3. Aufl. Pearson, 2006. ISBN: 978-0321455369.

A Anhang

A.1 Verzeichnis der portierten Lehrnotebooks

Die folgende Tabelle listet alle Jupyter-Notebooks auf, die im Rahmen dieser Arbeit selektiert und von Python nach TypeScript portiert wurden. Die Gliederung folgt der Kapitelstruktur der Originalvorlesung „Theoretische Informatik III“.

Tabelle 1: Übersicht der übersetzten Notebooks im Repository

Original-Kapitel	Notebook-Dateiname
Chapter 03	01-Ply-Scanning-Example.ipynb 02-Regex-Tutorial.ipynb 03-Exam-Evaluation.ipynb 04-Html2Text.ipynb
Chapter 04 & 05	01-NFA-2-DFA.ipynb 02-Test-NFA-2-DFA.ipynb 03-Regex-2-NFA.ipynb 04-Test-Regex-2-NFA.ipynb 05-DFA-2-RegExp.ipynb 06-Test-DFA-2-RegExp.ipynb 07-Minimize.ipynb 08-Test-Minimize.ipynb 09-Equivalence.ipynb 10-Test-Equivalence.ipynb
Chapter 06	01-Top-Down-Parser.ipynb 02-EBNF-Parser.ipynb
Chapter 07	01-Calculator.ipynb 02-Differentiator.ipynb 03-Interpreter.ipynb
Chapter 10	01-Conflicts.ipynb 02-Conflicts-Resolved.ipynb

Quelle: Eigene Darstellung.